

Kollisionskaskaden in 2D

Ein HPC-Projektbericht: GPU-Beschleunigung und Potenzgesetze

Paul Kamm

TH Lübeck – Kursprojekt *High Performance Computing*, betreut von Dr. Maurice Maurer
Messungen auf *Noctua 2*, PC2 Paderborn (EPYC 7763, A100-SXM4-40GB)

15. August 2026

Zusammenfassung

Dieser Bericht untersucht 2D-Kollisionskaskaden aus zwei Perspektiven: als **HPC Projekt** und als **physikalisches Skalierungsgesetz**. **Teil I** (HPC) verfolgt die Frage, wie weit sich eine Molekulardynamik-Simulation durch Parallelisierung und GPU-Portierung beschleunigen lässt – und wo die prinzipielle Grenze liegt. Es zeigt sich, dass der Code durchgehend *speicherbandbreiten-gebunden* ist: eine A100-GPU liefert **731-fache** Beschleunigung gegenüber einem EPYC-Kern; die MPI-Skalierung ist U-förmig mit superlinearer Effizienz (133 % bei 128 Kernen) durch Cache-Effekte; Kommunikationsoptimierungen bringen nichts. **Teil II** (Power Law) zeigt, dass die Clustergrößenverteilung aus 1000 simulierten Kaskaden einem Potenzgesetz mit $S = 1,38\text{--}1,41$ folgt – robust gegen Healing. Behandelt werden die Fit-Methodik (Maximum Likelihood nach Clauset et al., $x_{\min}$ -Falle) sowie die Einordnung des Ergebnisses in den Kontext der 3D-MD-Befunde von Sand et al. (2013) und der Subkaskaden-Theorie von De Backer et al. (2016).

Inhaltsverzeichnis

I	HPC – Beschleunigung der Simulation	3
1	Das Simulations-Modell	3
1.1	Physikalisches Modell: 2D-Dreiecksgitter	3
1.2	Datenstruktur: Structure of Arrays (SoA)	5
1.3	Ortsraum-Zerlegung: Cell List	5
1.4	Integrator: Velocity-Verlet	6
2	MPI-Parallelisierung	6
2.1	Zerlegungs-Strategie	6
2.2	Phasen im MPI-Zeitschritt	7
2.3	Speicherkorrektheit: Valgrind-Analyse	7
3	Compiler-Optimierungen (Einzelkern)	8
4	Strong Scaling – fester Problemumfang	9
5	Cache-Argument: Warum die Effizienz superlinear wird	10
6	1-Knoten vs. 2-Knoten-Vergleich	12
7	Weak Scaling – variable Problemgröße	12
8	Optimierungsvergleich: Non-Blocking Halo und Re-Sorting	13
9	CUDA-Portierung	14
9.1	Architektur-Entscheidungen	14
9.2	Benchmark-Ergebnis: A100-SXM4-40GB	14
10	GPU-Sättigung: Skalierung mit der Problemgröße	15
11	ncu-Profilng: Warum ist der A100 trotzdem kein Optimum?	16
11.1	SpeedOfLight: Compute vs. Memory	16
11.2	Okkupanz: Registerlimitierung	16
11.3	Experiment: Occupancy-Hebel via <code>__launch_bounds__</code>	17
11.4	Weitere Optimierungsversuche: Launch-Overhead und Präzision	18
12	Fazit Teil I	19
II	Power Law – Skalenfreie Defektverteilung	20
13	Theoretischer Hintergrund	20
13.1	Was ist ein Potenzgesetz?	20
13.2	Sand et al. (2013): Potenzgesetz in 3D-Wolfram	20
13.3	De Backer et al. (2016): Subkaskaden und das fragmentierte Regime	21
14	Ensemble-Setup	22
15	Fit-Methodik: Maximum-Likelihood-Estimation	22
15.1	Clauset et al. – MLE statt Histogramm-Fit	22
15.2	Die $x_{\min}$ -Falle	23
16	Ergebnisse: Power Law in 2D	23
17	Diskussion	25
17.1	Warum $S < 1,63$?	25
17.2	Robustheit gegen Healing	25
17.3	Verbindung zur De Backer-Theorie	25
17.4	Ausblick und offene Fragen	25
18	Machbarkeitsstudie: Skalierung des Ensembles	25
19	Fazit Teil II	27
A	Anhang: Skripte und Reproduzierbarkeit	27

Teil I

HPC – Beschleunigung der Simulation

1 Das Simulations-Modell

1.1 Physikalisches Modell: 2D-Dreiecksgitter

Simuliert wird kein echtes Material, sondern der *Mechanismus* einer Kollisionskaskade in einem minimalen 2D-Modell. Die Atome sitzen auf einem **Dreiecksgitter** (6er-Koordination, schubstabil, elastisch isotrop; Abb. 1).

Zwei Wechselwirkungen (Abb. 2).

- **Brechbare harmonische Feder:** $F = K(r - L_0)$ mit Bruch bei $r > 1,15 L_0$ ($K = 100$). Modelliert die kovalente/metallische Bindung.
- **Steile Kurzreichweitenswechselwirkung:** $F = K_{\text{rep}}((R_{\text{cut}}/r)^{12} - 1)$ mit $K_{\text{rep}} = 400$, $R_{\text{cut}} = 0,9 L_0$. Gibt den Atomen einen effektiven Wirkungsquerschnitt – ohne sie würde das PKA alles durchdringen.

Das PKA (Primary Knock-on Atom) ist schlicht ein Atom mit einer hohen Anfangsgeschwindigkeit $v_{\text{PKA}} = \sqrt{2E_{\text{PKA}}/m}$, die in eine Richtung zeigt. Im Laufe der Simulation überträgt es seine Energie durch Abstoßungen auf Nachbaratome, die ihrerseits weitere anstoßen – die verzweigende Kaskade (Abb. 3).

M1 -- Modell-Schema: 2D-Dreiecksgitter mit brechbaren Federn

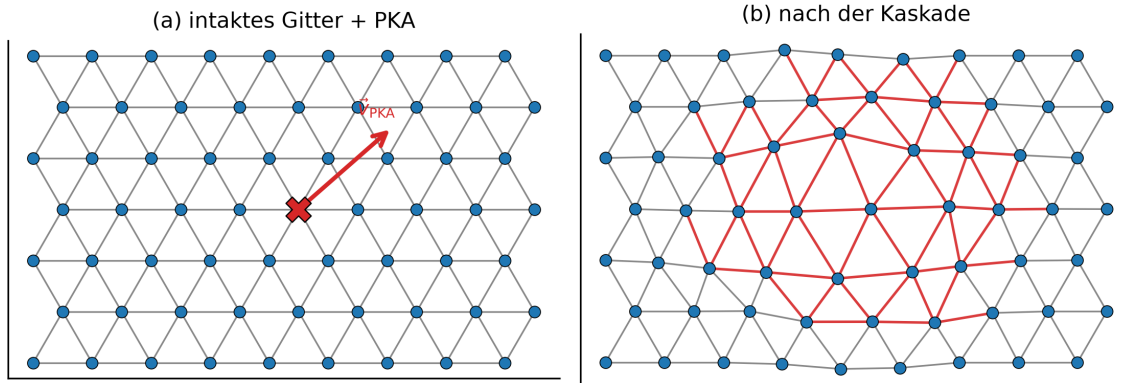


Abbildung 1: Modell-Schema. Links: intaktes Dreiecksgitter mit rot markiertem PKA. Rechts: Zustand nach der Kaskade – gerissene Bindungen (rot) und ein Feld aus Defektclustern. Alle Größen sind dimensionslos.

M2 -- Die beiden Wechselwirkungen (echte Code-Parameter)

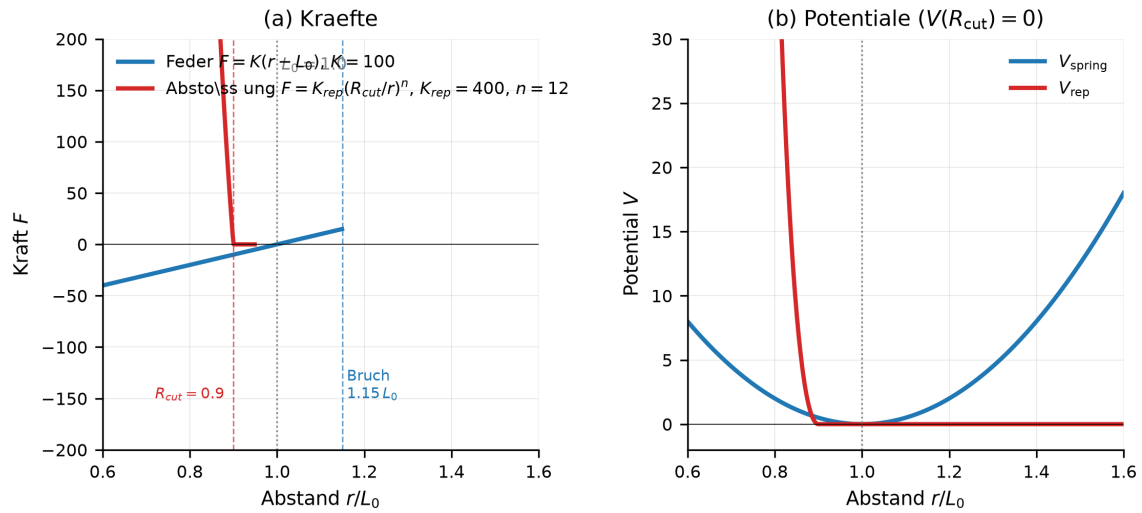


Abbildung 2: Die beiden Wechselwirkungen. Links: Kraftverläufe – Feder (blau, Bruch bei $1,15 L_0$) und Abstoßung (rot, nur für $r < 0,9 L_0$). Rechts: zugehörige Potentiale mit stetigem Nullpunkt an R_{cut} . Die Energiestetigkeit ist Grundlage des Korrektheitschecks.

M4 -- PKA-Geschwindigkeitszerlegung

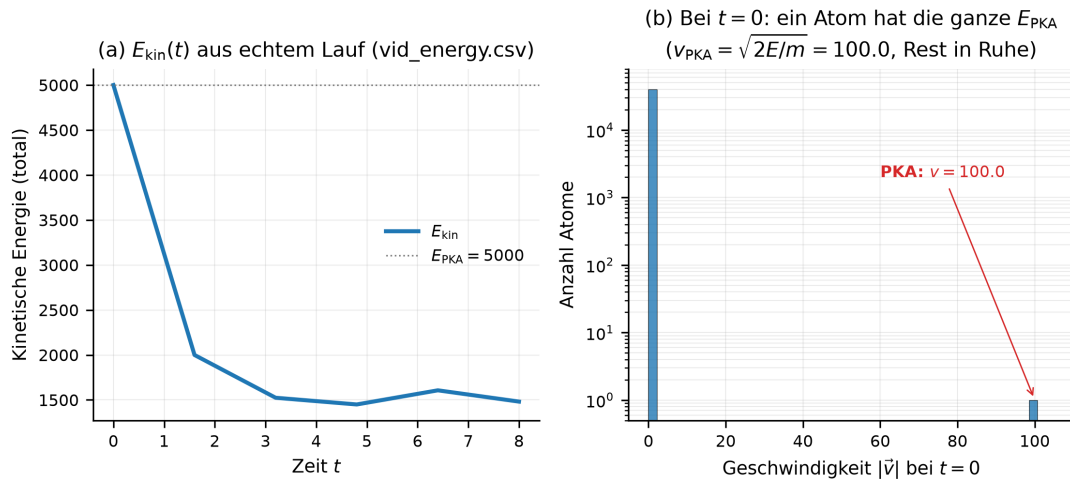


Abbildung 3: PKA-Start (Korrektheitsprüfung). Links: kinetische Gesamtenergie über die Zeit – bei $t = 0$ steckt sie vollständig im PKA-Atom und wird im Zuge der Kaskade auf die Nachbaratome verteilt; rechts: Geschwindigkeitshistogramm bei $t = 0$, ein einzelner Peak bei $v_{PKA} = \sqrt{2E/m} \approx 100$, alle anderen Atome stehen still. Genau dieser Impuls startet die Kaskade.

1.2 Datenstruktur: Structure of Arrays (SoA)

Die Atome werden als **Structure of Arrays** gespeichert: statt `atom[i].px` gibt es separate Arrays `px[i]`, `py[i]`, `vx[i]`, `vy[i]`, `fx[i]`, `fy[i]`, `nbr[i][6]`, `cell[i]`.

Speicherlayout (76 B/Atom)

8 Felder: `px, py, vx, vy, fx, fy` (je $8\text{ B} \times 6$) + `nbr` ($4\text{ B} \times 6$, sechs Nachbarn im Dreiecksgitter) + `cell` (4 B) = 76 B/Atom . Bei $N = 2\text{ Mio}$: Gesamtdatensatz $\approx 152\text{ MB}$. Dieses Maß wird in Abschnitt 5 zur Cache-Analyse verwendet.

SoA hat drei Vorteile für diese Simulation:

1. **Cache-Effizienz:** Der Kraftkern iteriert über N Atome und braucht dabei px , py , nbr sequenziell – bei SoA ist das ein einziger linearer Speicherzugriff.
2. **Vektorisierbarkeit:** SIMD-Einheiten (AVX-512 auf EPYC) können mehrere Atome gleichzeitig laden und rechnen.
3. **GPU-Kompatibilität:** CUDA-Kernel können mit einem Thread pro Atom arbeiten und koaleszente Speicherzugriffe machen (Abschnitt 9).

1.3 Ortsraum-Zerlegung: Cell List

Ohne weitere Maßnahme wäre die Komplexität des Kraftkerns $O(N^2)$ – für 2 Mio Atome nicht praktikabel. Die **Cell List** löst das (Abb. 4):

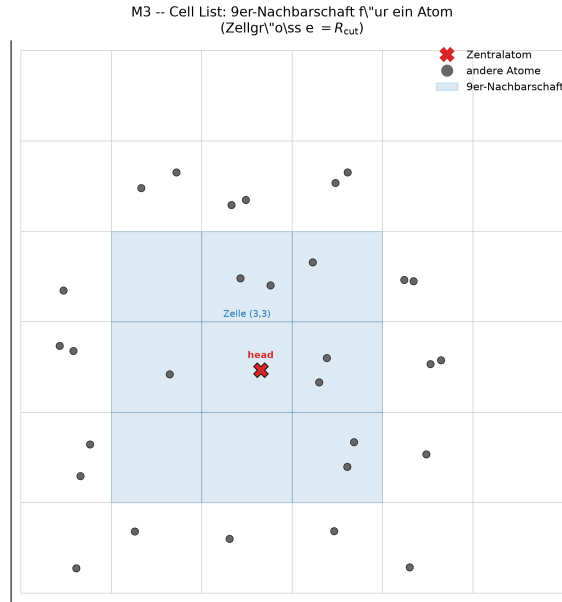


Abbildung 4: Cell-List-Schema. Der Raum wird in Zellen der Größe R_{cut} unterteilt. Für ein Atom (rot) genügt es, die 9 Nachbarzellen (blaues Band) zu durchsuchen. Verkettete Listen über `cell_head/cell_next` machen die Nachbarsuche $O(N)$ statt $O(N^2)$.

Jede Zelle hat Seitenlänge $\geq R_{\text{cut}}$; potenzielle Bindungspartner liegen garantiert in den 9 umgebenden Zellen. Für N Atome mit konstanter Dichte gilt: mittlere Atome pro Zelle $= O(1)$, also Gesamtkomplexität $O(N)$. Die Cell List muss nach jeder Migration neu aufgebaut werden, was im MPI-Zeitschritt als eigenständige Phase (`cells`) gemessen wird.

1.4 Integrator: Velocity-Verlet

Die Bewegungsgleichungen werden mit **Velocity-Verlet** in Kick-Drift-Kick-Form integriert:

$$\mathbf{v}_i^{n+1/2} = \mathbf{v}_i^n + \frac{\Delta t}{2} \mathbf{F}_i^n / m \quad (\text{Kick}) \quad (1)$$

$$\mathbf{r}_i^{n+1} = \mathbf{r}_i^n + \Delta t \mathbf{v}_i^{n+1/2} \quad (\text{Drift}) \quad (2)$$

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^{n+1/2} + \frac{\Delta t}{2} \mathbf{F}_i^{n+1} / m \quad (\text{Kick}) \quad (3)$$

Als symplektischer Integrator erhält Velocity-Verlet die Phasenraum-Struktur und zeigt keinen systematischen Energiedrift – im Gegensatz zum expliziten Euler-Verfahren. Abbildung 5 belegt, dass die Gesamtenergie über 3000 Schritte unter 0,6 % (seriell) bzw. 0,22 % (A100, fp64) driftet. Abbildung 6 zeigt die zugehörige Zeitentwicklung der Kaskade.

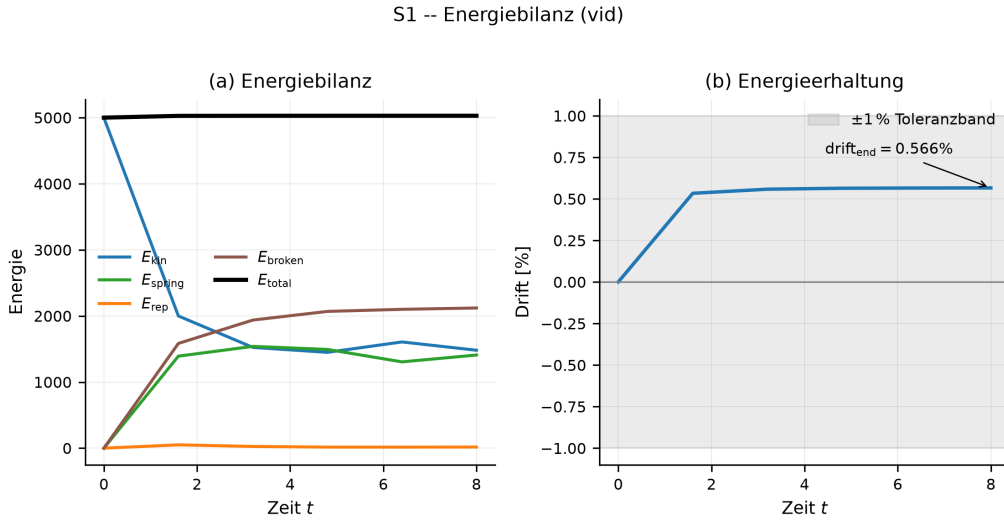


Abbildung 5: Energiebilanz (Korrektheitsprüfung). Die Gesamtenergie E_{total} (dicke schwarze Linie) bleibt praktisch konstant. Der prozentuale Drift liegt unter 0,6 % für die serielle CPU-Version und unter 0,22 % für den A100 (fp64). Ohne diesen Nachweis wäre jede Performance-Messung wertlos.

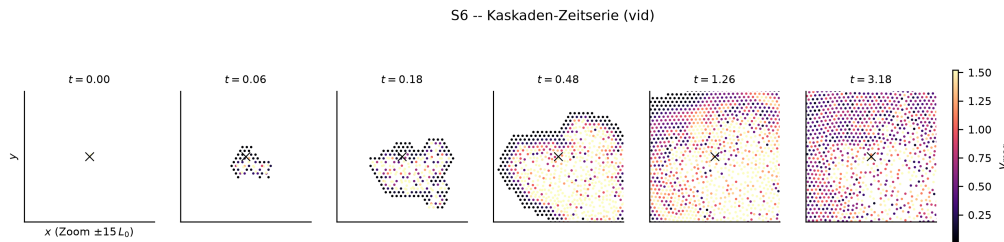


Abbildung 6: Kaskaden-Zeitentwicklung ($t = 0 \dots T$). Farbe = Geschwindigkeitsbetrag. Ballistische Phase (Ausbreitung der Stoßfront) klingt ab $t \sim 1$ ab; danach steht die Defektstruktur. Die Zeitskala $T = 3000 \Delta t$ ist groß genug, damit die Kaskade vollständig zur Ruhe kommt.

2 MPI-Parallelisierung

2.1 Zerlegungs-Strategie

1D-Domänenzerlegung in y -Richtung. Der 2D-Raum wird in horizontale Streifen zerlegt, jeweils ein Streifen pro MPI-Rank. Die 1D-Zerlegung hat einen entscheidenden Vorteil: von den $P - 1$ Rank-Grenzen kreuzt beim Netzwerkwechsel (z.B. von Knoten 1 zu Knoten 2) genau eine einzige – die Inter-Node-Kommunikation bleibt minimal, wie Abschnitt 6 belegt.

Gather-Formulierung. Jedes Atom summiert *nur seine eigene Kraft*, Newton 3 wird nicht genutzt. Das bedeutet: jedes Atompaar (i, j) wird zweimal ausgewertet. Diese scheinbar ineffiziente Formulierung hat drei Gründe:

1. **Korrektheit über Rank-Grenzen:** Ein Geisteratom (Kopie aus dem Nachbar-Rank) hat keinen “Besitzer”, der die Hälfte der Kraft einsammeln würde.
2. **Atomicity-Freiheit auf der GPU:** Kein `atomicAdd` nötig (Abschnitt 9).
3. **Cache-Effizienz:** Jeder Thread schreibt nur an *einen* Speicherplatz.

2.2 Phasen im MPI-Zeitschritt

Jeder MPI-Zeitschritt besteht aus sechs messbaren Phasen:

Phase	Aufgabe	Typ	Anteil @ 1 Kern
force	Federn + Abstoßung + Bindungsbruch (Gather)	Rechnung	89 %
hash	gid $\rightarrow$ lokaler Index (für Bindungsbruch)	Rechnung	6 %
cells	Cell List neu aufbauen (lokal + Halo)	Rechnung	2 %
halo	Geisteratome von Nachbar-Ranks kopieren	Komm.	<1 %
migrate	Besitzwechsel beim Grenzübertritt	Komm.	<1 %
energy	Globale Energiediagnose (MPI_Reduce)	Diagnose	<1 %

Halo-Austausch. Jeder Rank sendet eine Kopie seiner Randatome (“Geisteratome”) an die beiden Nachbar-Ranks. Das passiert jeden Zeitschritt, weil die Federn über Ranggrenzen hinweg wirken können. Das Halo-Volumen ist proportional zur Gitterbreite N_x (konstant bei 1D-Zerlegung), was den Kommunikationsanteil bei kleinen Rank-Zahlen harmlos hält, aber bei vielen Ranks relativ wächst.

Atom-Migration. Überquert ein Atom seine Rank-Grenze, wechselt es den “Besitzer”. Das Atom wird mit allen Daten (Position, Geschwindigkeit, Bindungstabelle) an den neuen Rank gesendet und beim alten gelöscht. In einer Kaskade betrifft das typischerweise nur wenige Atome pro Schritt.

2.3 Speicherkorrektheit: Valgrind-Analyse

Neben der physikalischen Korrektheit (Energieerhaltung, Abb. 5) prüft *Valgrind Memcheck* zusätzlich die Speicherkorrektheit beider CPU-Implementierungen: ungültige Zugriffe, Verwendung uninitialisierten Speichers und Speicherlecks. Da Memcheck jeden Speicherzugriff instrumentiert und dadurch 20–50 $\times$ langsamer läuft, wurde nicht das 2-Mio.-Atome-Benchmarkgitter verwendet, sondern ein kleines Gitter ($80 \times 80 = 6400$ Atome, 300 Schritte, `params_valgrind.ini`). Dieses durchläuft dieselben Code-Pfade (Allokation, Kraftberechnung, Cell-List, Halo-Austausch, Migration), ist aber in Sekunden statt Stunden analysiert.

```
# Serielle Version (mit gcc gebaut, damit keine MPI-Bibliothek stoert):
gcc -O1 -g -lm -march=native -o cascade_serial cascade_serial.c
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes \
    ./cascade_serial params_valgrind.ini

# MPI-Version (2 Raenge, ein Logfile pro Rang):
mpicc -O1 -g -lm -march=native -o mpi_cascade mpi_cascade.c
mpirun --oversubscribe -np 2 valgrind --leak-check=full \
    --show-leak-kinds=definite,indirect --track-origins=yes \
    --log-file=vg_rank_%{OMPI_COMM_WORLD_RANK}.txt \
    ./mpi_cascade params_valgrind.ini
```

Kategorie	cascade_serial	mpi_cascade (je Rang)
Ungültige Zugriffe (App)	0	0
Uninit. Werte (App)	0	0
<i>definitely lost</i> (App)	0	$\approx 1,22$ MB
<i>still reachable</i> (App)	$\approx 0,96$ MB	—
Fehler in MPI-Bibliothek	—	$\sim 10^7$ (Fehlalarme)

Serielle Version: sauber. Memcheck meldet *0 errors from 0 contexts*. Die $\approx 0,96$ MB *still reachable* sind die einmalig allokierten SoA-Arrays, die bewusst bis Programmende gehalten werden; alle Allokationen pro Zeitschritt werden korrekt freigegeben.

MPI-Version: keine echten Fehler im Anwendungscode. Die $\sim 10^7$ gemeldeten Fehler stammen *sämtlich* aus dem Inneren von OpenMPI (MPI_Init) – ein bekannter, harmloser Valgrind-Fehlalarm; kein einziger Fehler-Kontext zeigt auf `mpi_cascade.c`. Die $\approx 1,22$ MB *definitely lost* sind die einmalig zum Start allokierten Strip-SoA-Felder, deren Zeiger beim Leak-Check (nach Rückkehr aus `main`) nicht mehr erreichbar sind. Es ist *kein* mit der Laufzeit wachsendes Leck: den 44 386 Allokationen stehen 43 730 Freigaben gegenüber – die Differenz sind genau die Startup-Felder.

Schlüsselergebnis Valgrind

Beide CPU-Versionen sind frei von ungültigen Speicherzugriffen und Verwendung uninitialisierter Werte im Anwendungscode. Es gibt *kein* mit der Laufzeit wachsendes Speicherleck; sämtliche nicht freigegebene Speicher sind einmalige Startup-Allokationen, die das Betriebssystem beim Programmende zurückgibt. Die Fehlermeldungen der MPI-Version sind Bibliotheks-interne Valgrind-Fehlalarme aus MPI_Init.

3 Compiler-Optimierungen (Einzelkern)

Vor der Parallelisierung stellt sich die Frage: Wie viel bringt der Compiler? Gemessen mit verschiedenen Flag-Kombinationen auf einem EPYC-7763-Kern.

Schlüsselergebnis Compiler

Baseline -O0: 113,7 s. Schnellstes *korrektes* Setup: -O3 mit 57,0 s ($\approx 2,0\times$). -Ofast/-ffast-math ist *langsamer* und verletzt die Energieerhaltung.

H9 -- Compiler-Optimierungen (Speedup vs. O0, Drift $\leq 0.22\%$ bei allen)

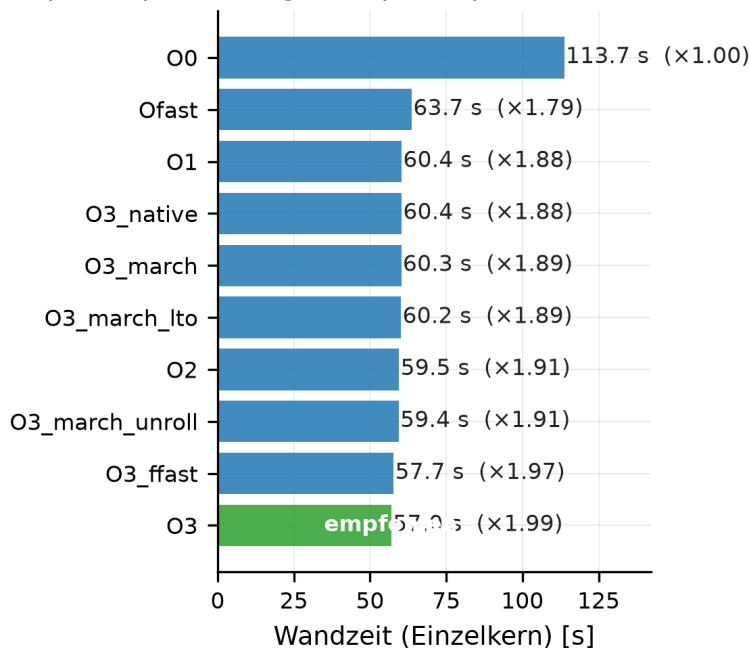


Abbildung 7: Compiler-Optimierungen auf einem EPYC-7763-Kern (Noctua 2), sortiert nach Wandzeit. Grün markiert ist -O3 als empfohlenes Setup. -O3 liefert $\approx 2\times$ Speedup gegenüber -O0. -Ofast und -ffast-math umgehen IEEE-754-Regeln und erzeugen nicht-assoziative Floating-Point-Operationen, was in einem symplektischen Integrator grundsätzlich riskant für die Energieerhaltung ist.

Warum bringt -03 nur den Faktor 2? Die **force**-Phase ist *speicherbandbreiten-gebunden*: sie streamt ~ 152 MB Atom-Daten pro Schritt und rechnet dabei pro Atom nur wenige Floating-Point-Operationen. Bessere Instruktionen helfen nicht, wenn die Daten nicht schnell genug ankommen. Dieses Muster zieht sich wie ein roter Faden durch alle folgenden Messungen.

4 Strong Scaling – fester Problemumfang

Setup: $N = 1\,999\,396$ Atome (1414×1414), 1–256 Kerne, Phasentimer aktiv, ein Lauf je Rankzahl (Wiederholungsmessungen mit identischem Setup liefert der Optimierungsvergleich in Abschnitt 8).

H1 -- Strong Scaling (Speedup & Effizienz)

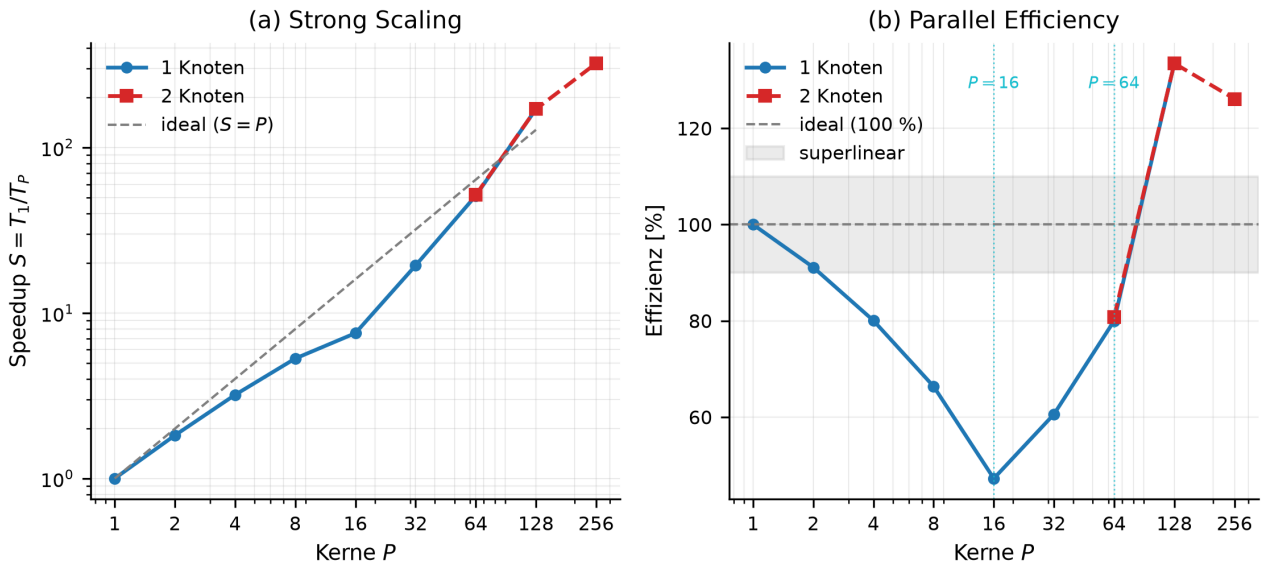


Abbildung 8: Strong Scaling. Oben: Speedup $S = T(1)/T(P)$ vs. P , mit idealer Gerade (grau). Unten: Effizienz S/P in %. Die U-Form mit Minimum bei 16 Kernen und superlinearem Maximum (133 %) bei 128 Kernen ist kein Messfehler – Abschnitt 5 erklärt sie quantitativ durch Cache-Effekte.

Schlüsselzahlen.

Kerne	$T(P)$ [s]	Speedup	Effizienz
1	1152,9	1,0	100 %
4	360,1	3,2	80 %
8	217,2	5,3	66 %
16	152,6	7,6	47 % (Minimum)
32	59,5	19,4	61 %
64	22,5	51,2	80 %
128	6,7	170,8	133 % (superlinear!)

Drei Regionen der Effizienzkurve.

- **1→16 Kerne (Abstieg):** Die 1D-Zerlegung fügt bei jedem neuen Rank eine Halo-Grenze hinzu. Das Halo-Volumen pro Rank bleibt konstant (volle Breite N_x), während die Rechenarbeit pro Rank schrumpft. Bei 16 Kernen liegt der Kommunikationsanteil (Maximum über die Ranks) bei ≈ 35 % der Wandzeit; er wächst bis 32 Kerne weiter auf ≈ 60 % (Abb. 10), wird dort aber bereits von den einsetzenden Cache-Effekten überkompensiert.
- **16→64 Kerne (Erholung):** Cache-Effekte setzen ein: die Datenmenge pro Rank sinkt unter ≈ 9 MB und nähert sich dem L3-Cache (4 MB/Rank).

- **64→128 Kerne (superlinear):** Datensatz/Rank liegt weit unterhalb des L3-Limits – der *force*-Kernel wird cache-resident. Der Speedup übersteigt das Ideal. Das ist quantitativ vorhergesagt durch das Cache-Argument (Abschnitt 5).

5 Cache-Argument: Warum die Effizienz superlinear wird

AMD EPYC 7763 L3-Cache

AMD EPYC 7763: 8 Cache Dies (CCDs), je 32 MB L3, je 8 Kerne. $\Rightarrow$ L3-Anteil pro Rank: $32 \text{ MB}/8 = 4 \text{ MB}$. SoA-Datensatz: $76 \text{ B/Atom} \times N/P$. Bei $P = 64$ Ranks: $76 \times 1,999 \cdot 10^6 / 64 \approx 2,4 \text{ MB} < 4 \text{ MB} \Rightarrow$ cache-resident.

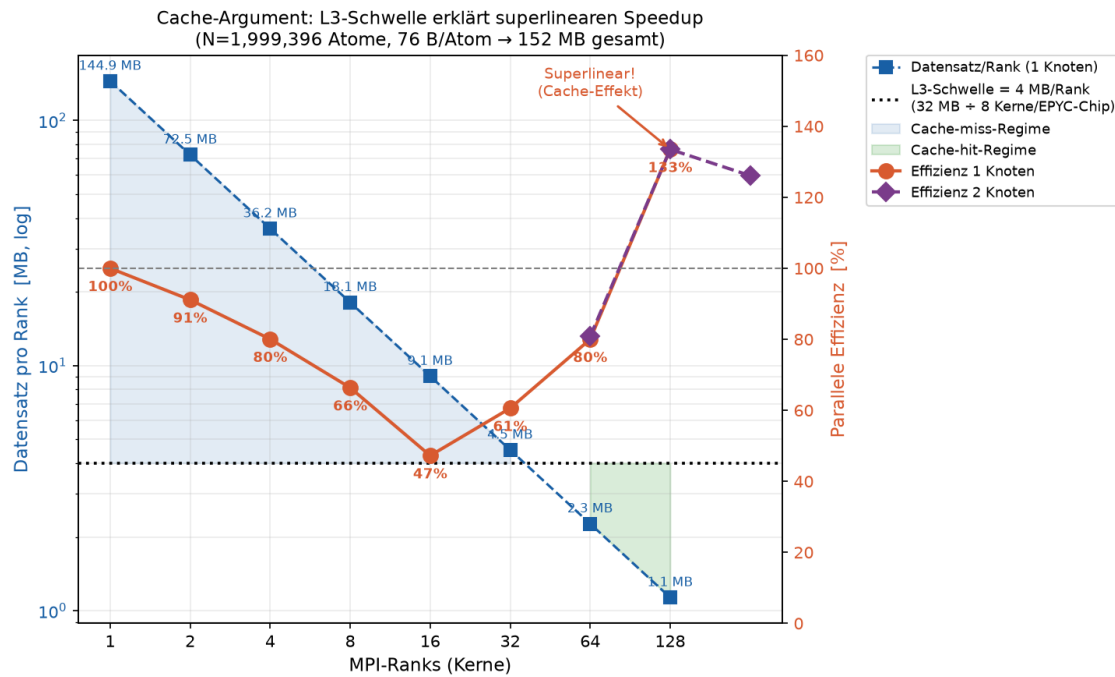


Abbildung 9: Cache-Argument (Dual-Achsen-Plot). Blaue Kurve (linke Achse, log): Datensatz pro Rank in MB, sinkt mit $1/P$. Gestrichelte Linie: L3-Schwelle bei 4 MB/Rank. Rote Kurve (rechte Achse): parallele Effizienz. Der Datensatz unterschreitet die L3-Schwelle zwischen 32 und 64 Ranks; die Effizienz erholt sich dort und übersteigt bei 128 Ranks 100 %. Skript: `scripts/python/plot_cache_argument.py`.

Der Mechanismus ist direkt: Die *force*-Phase streamt die Arrays `px`, `py`, `nbr` für jedes Atom. Solange der Datensatz nicht in den L3-Cache passt, muss er bei jedem Schritt aus dem DRAM geladen werden (Bandbreite: ca. 400 GB/s pro EPYC-Knoten, geteilt unter allen Kernen). Sobald er in den L3 passt, ist er cache-resident: der Kernel sieht $\sim 2 \text{ TB/s}$ effektive Bandbreite – also $5\times$ so viel. Genau das erklärt den superlinearen Sprung.

H2 -- Phasen-Breakdown

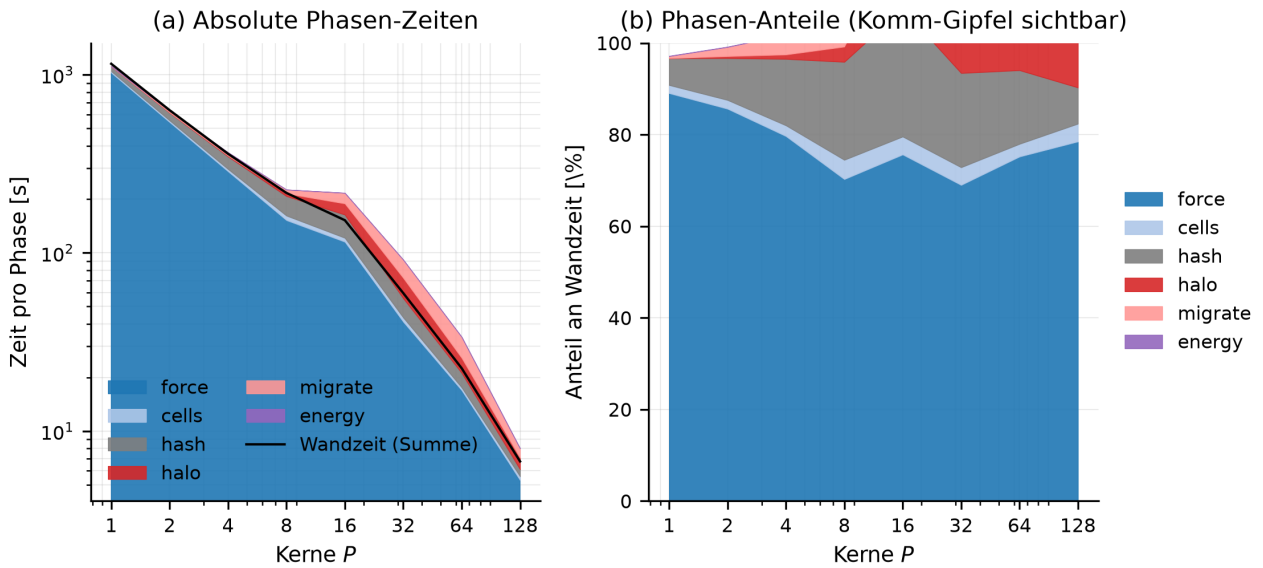


Abbildung 10: Phasenbreakdown über alle Kerne. Links: absolute Zeiten (log- y) – **force** (blau) fällt überproportional. Rechts: normierter Stack – der Kommunikationsanteil (**halo+migrate**, Rottöne) wächst mit der Rankzahl deutlich an, während **force** dominant bleibt.

H3 -- Rechnen vs. Kommunikation

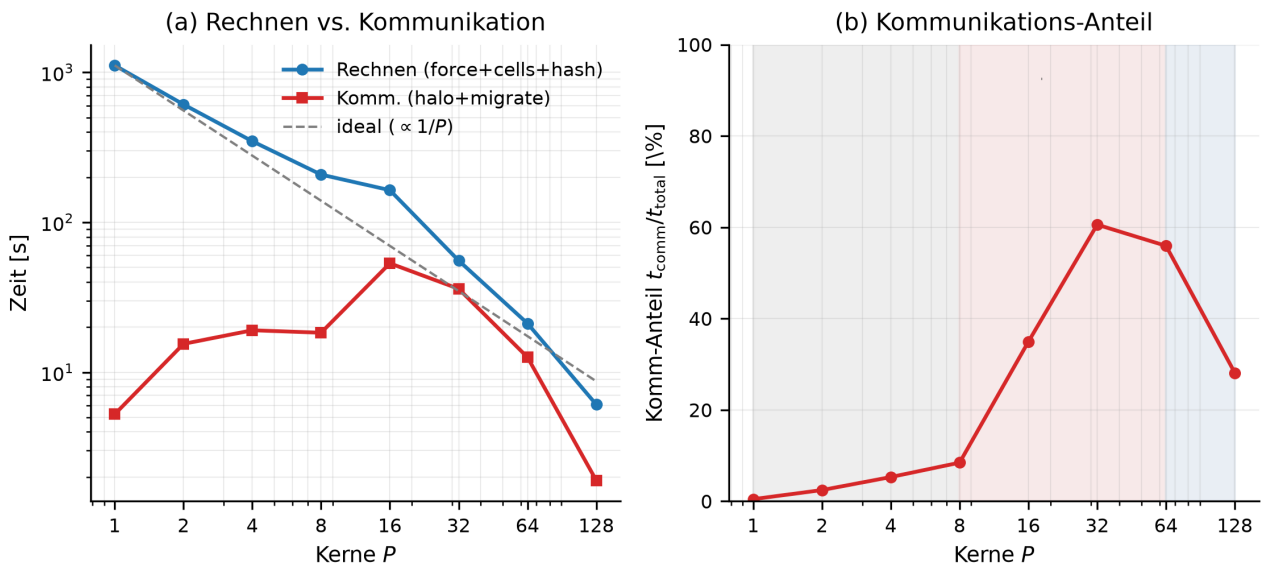


Abbildung 11: Rechnen vs. Kommunikation. Rechenzeit (force+cells+hash, blau) fällt schneller als $1/P$ – superlinear wegen Cache. Kommunikationszeit (halo+migrate, rot) wächst zunächst und sinkt erst bei voller Knotenbelegung. Der Flaschenhals wandert mit der Rankzahl.

6 1-Knoten vs. 2-Knoten-Vergleich

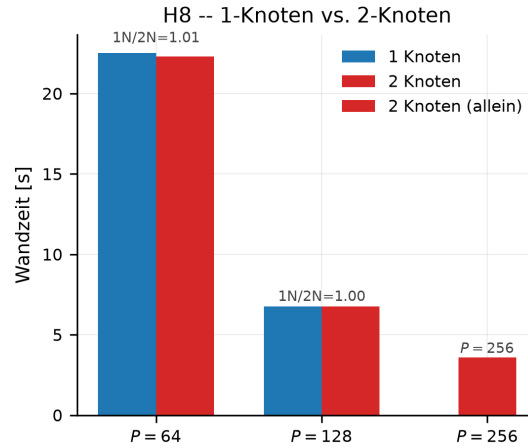


Abbildung 12: 1-Knoten vs. 2-Knoten. Bei überlappenden Rank-Zahlen ($P = 64, 128$) sind die Wandzeiten praktisch identisch. In der 1D-Zerlegung kreuzt genau eine von $P - 1$ Rank-Grenzen den Inter-Node-Link – die Netzwerkkosten sind vernachlässigbar.

Der 256-Kerne-Punkt (nur 2-Knoten erreichbar) zeigt, dass die Superlinearität auch über Knoten-Grenzen hinweg anhält. Die 1D-Zerlegung ist robust gegenüber der Netzwerktopologie: von den $P - 1$ Rank-Grenzen sind $(P - 2)$ intra-node und genau eine inter-node.

7 Weak Scaling – variable Problemgröße

Setup: Problemgröße wächst proportional zur Rankzahl (konstante Atomzahl pro Rank). Im Idealfall konstante Wandzeit.

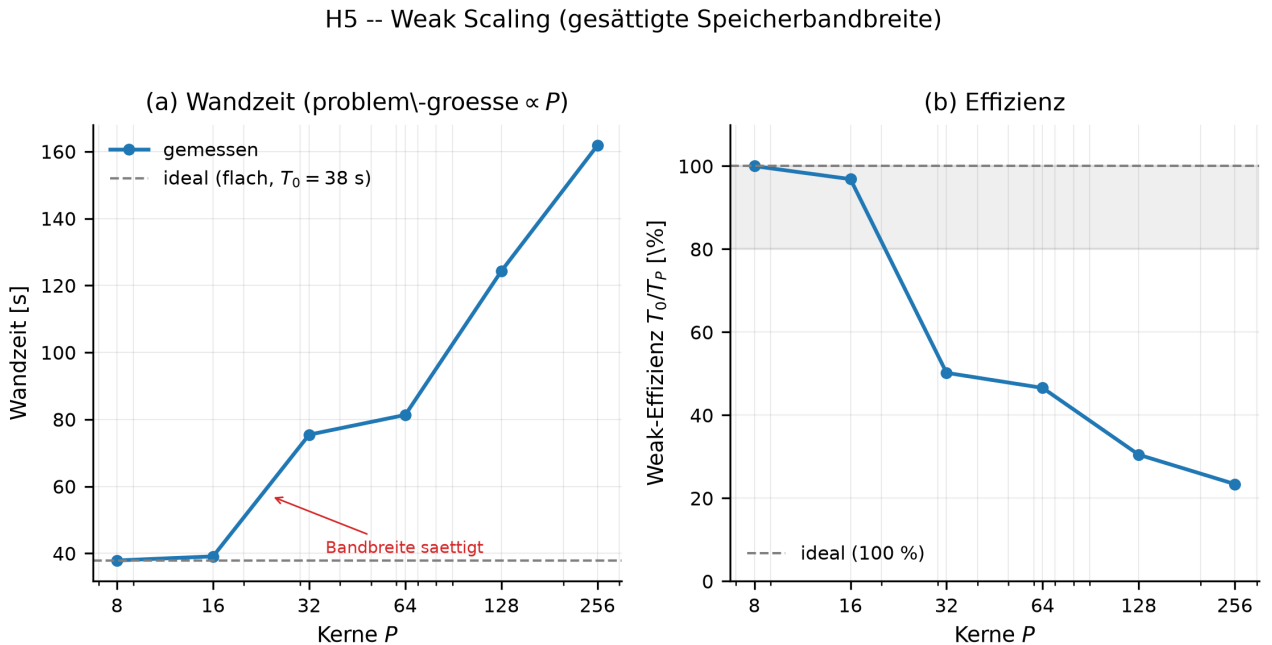


Abbildung 13: Weak Scaling. Links: Wandzeit – bis 16 Kerne nahezu ideal, dann monoton steigend auf ≈ 162 s bei 256 Kernen. Rechts: Weak-Effizienz T_0/T_P – sinkt auf 23 %.

Mechanismus: gesättigte Speicherbandbreite. Im Weak Scaling bleibt die Datenmenge *pro Rank* konstant – aber die *gemeinsame* Speicherbandbreite eines Knotens ist physikalisch fest und wird von allen Kernen geteilt. Bis 16 Kerne ist sie nicht ausgelastet (flache Kurve). Ab ≈ 32 Kernen ist sie gesättigt – mehr Kerne liefern mehr Rechenwerke, aber nicht mehr Speicherdurchsatz. Jeder weitere Kern wartet auf Daten.

Einheitliche Diagnose

Strong Scaling: superlinear wegen Cache. Weak Scaling: bricht ein wegen Speicherbandbreite. **Beide Befunde zeigen denselben Engpass aus verschiedenen Blickwinkeln:** der Code ist memory-bound, nicht compute-bound und nicht kommunikationsgebunden.

8 Optimierungsvergleich: Non-Blocking Halo und Re-Sorting

Zwei klassische HPC-Optimierungen wurden als Schalter implementiert und gemessen:

- **Non-blocking Halo:** MPI_Isend/Irecv mit Richtungs-Overlap – beide Austauschrichtungen laufen gleichzeitig statt nacheinander (kein Überlappen mit Rechnung).
- **Periodisches Re-Sorting:** Umsortieren der SoA-Arrays nach räumlicher Lokalität – Cache-Hygiene nach Atom-Migration.

H6 -- Optimierungs-Vergleich (resort + non-blocking)

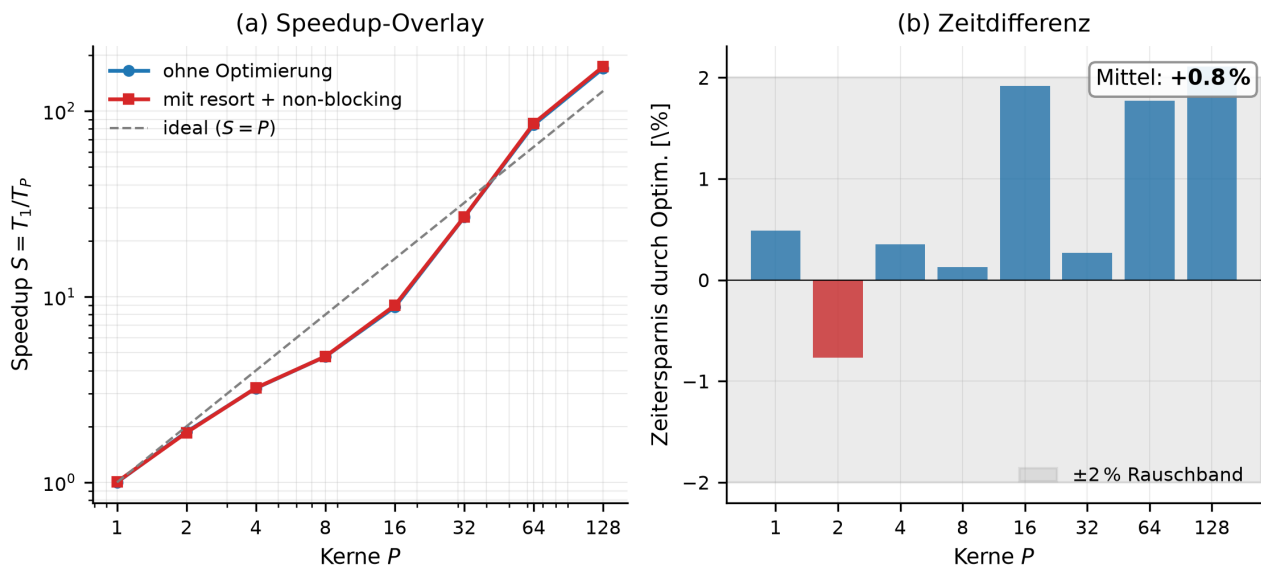


Abbildung 14: Optimierungsvergleich über 1–128 Kerne. Links: Speedup-Kurven ohne (blau) und mit Optimierung (rot) liegen praktisch deckungsgleich. Rechts: relative Zeitersparnis liegt im $\pm 2\%$ -Rauschband, im Mittel $+0,8\%$.

Warum beide nichts bringen.

- **Non-blocking Halo halbiert bestenfalls die Zeit der beiden Austauschrichtungen.** Das ausgetauschte Datenvolumen ist klein; die gemessene *halo*-Zeit ist weit größer, als das reine Volumen erklären würde – sie ist überwiegend Warte-/Synchronisationszeit auf den Nachbar-Rank, und dieses Warten verschwindet durch *Isend/Irecv* nicht.
- **Re-Sorting greift nur bei starker Cache-Degradation durch Migration.** In einer Kaskade ist die Bewegung räumlich lokalisiert – kaum ein Atom überquert eine Rank-Grenze. Die Speicherordnung degradiert kaum; Re-Sorting kostet fast genauso viel wie es spart.

Diese Null-Ergebnisse sind das eigentliche Ergebnis, kein Versagen. Sie beweisen unabhängig von den Scaling-Kurven: der Flaschenhals ist die Speicherbandbreite.

9 CUDA-Portierung

9.1 Architektur-Entscheidungen

Die GPU-Version portiert dieselbe Gather-Formulierung auf CUDA: **ein Thread pro Atom**, $i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$.

Keine Atomics im Kraftkern. Da jeder Thread nur die Kraft auf *sein* Atom berechnet, gibt es keinen konkurrierenden Schreibzugriff. Jedes Paar (i, j) wird zweimal ausgewertet (wie in der CPU-Version), aber dafür keine `atomicAdd`-Operationen. Atomics sind auf GPU sehr teuer; die doppelte Rechenarbeit ist billiger als eine Synchronisation auf dem gemeinsamen Datenpfad.

Block-Größe 256. Aus dem Sweep über Block-Größen (128, 256, 512, 1024) ergibt sich 512 als leicht bestes und 256 als praktisch gleichwertig. Verwendet wird $T = 256$ (gut teilbar, gute Auslastung bei $N = 2$ Mio Atomen: 7811 Blöcke).

9.2 Benchmark-Ergebnis: A100-SXM4-40GB

A100 (fp64) Headline-Ergebnis

$N = 1\,999\,396$ Atome, 3000 Schritte, 5 Wiederholungen. Median: **1,5765 s = 525 μ s/Schritt = 3805 Matom/s**.

Force-Kernel: 259 μ s/Aufruf (48 % der Zeitschleife).

Energie-Drift: 0,22 % (fp64, symplektisch).

H2D-Transfer (184 MB): 14 ms (14 GB/s).

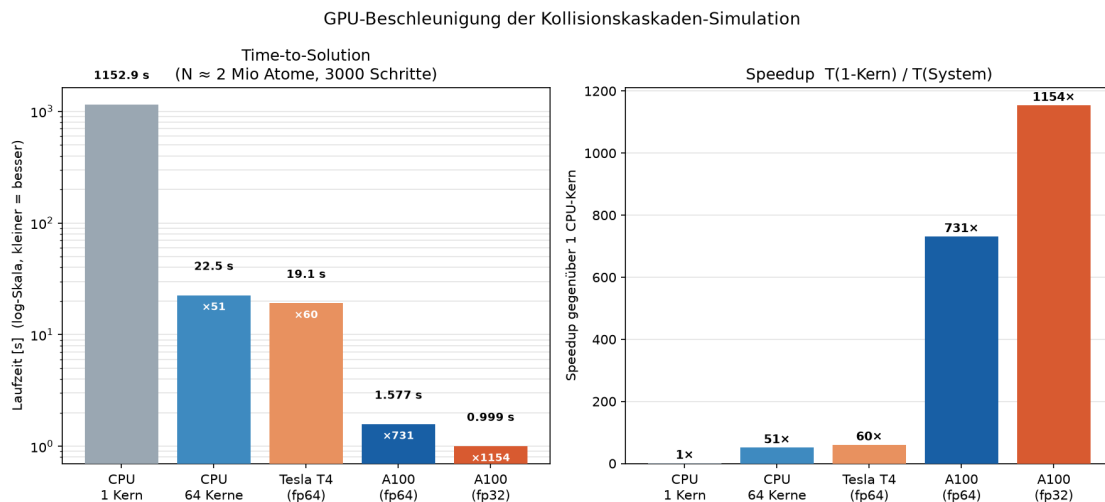


Abbildung 15: Time-to-Solution-Vergleich ($N \approx 2$ Mio Atome, 3000 Schritte). Links: Wandzeit auf Log-Skala; Rechts: Speedup relativ zu 1 CPU-Kern. Der A100 (fp64) ist **731-fach** schneller als 1 EPYC-Kern und **14-fach** schneller als 64 EPYC-Kerne. Skript: `scripts/python/plot_tts.py`.

fp32 vs. fp64. Bei fp32 halbiert sich die zu bewegende Datenmenge – ein bandbreitengebundener Kernel kann also theoretisch $2\times$ schneller laufen. Resultat: fp32 läuft in 0,999 s ($\approx 1,6\times$ schneller als fp64), aber der Energie-Drift steigt auf 0,94 % – noch tolerierbar für Ensemble-Statistik, aber grenzwertig für präzise Einzelkaskaden-Analyse.

Tesla T4 zum Vergleich. Auf der Tesla T4 läuft dieselbe Konfiguration in 19,09s – kaum schneller als die 64 EPYC-Kerne (22,5s). Der Grund ist die stark beschnittene fp64-Rechenleistung der T4 (Turing, sm_75): $\approx 0,25$ TFLOPS (1/32 der fp32-Leistung) gegenüber 9,7 TFLOPS fp64 des A100.

System	T [s]	Speedup vs. 1 Kern	Speedup vs. 64 Kerne
CPU 1 Kern	1152,9	1,0×	–
CPU 64 Kerne	22,5	51×	1,0×
Tesla T4	19,1	60×	1,2×
A100 fp64	1,58	731×	14×
A100 fp32	0,999	1154×	23×

10 GPU-Sättigung: Skalierung mit der Problemgröße

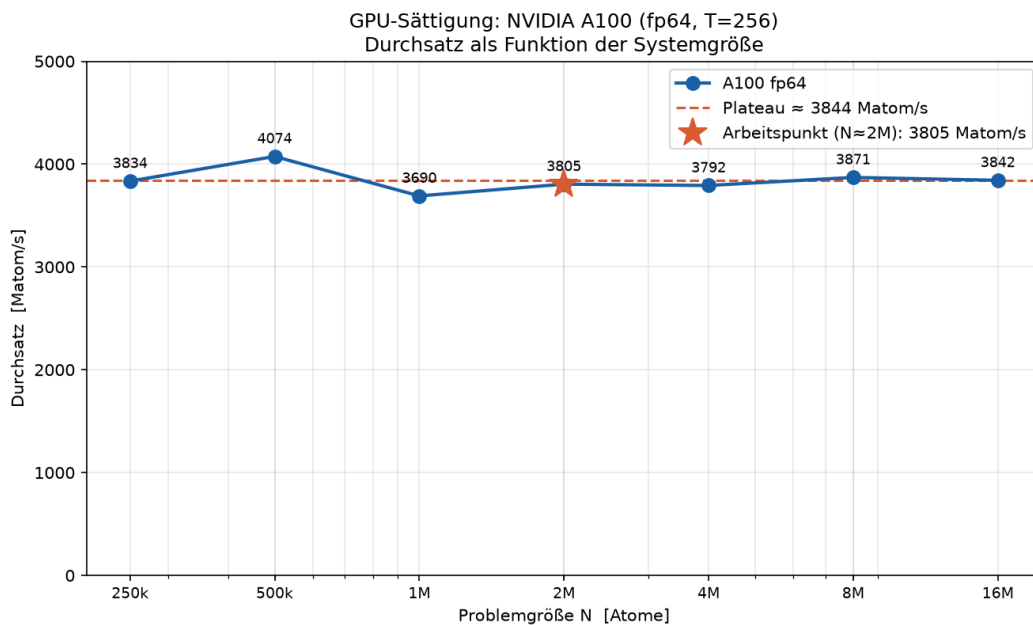


Abbildung 16: GPU-Sättigungskurve: Durchsatz in Matom/s als Funktion der Systemgröße N (A100, fp64). Bereits ab $N \approx 250\,000$ Atomen ist der Durchsatz konstant auf ≈ 3800 Matom/s – die GPU hat genug Threads um alle SMs vollständig auszulasten. Skript: `scripts/python/plot_gpu_saturation.py`.

Die Plateaustruktur belegt, dass die GPU ab $N \approx 250\,k$ voll gesättigt ist. Für $N < 250\,k$ reichen die Threads nicht aus, um alle 108 SMs des A100 zu füllen. Für $N > 250\,k$ skaliert die Laufzeit linear mit N – der Durchsatz ist konstant.

11 ncu-Profilng: Warum ist der A100 trotzdem kein Optimum?

11.1 SpeedOfLight: Compute vs. Memory

NVIDIA Nsight Compute (ncu) misst den Kernel als Prozentsatz des theoretischen Peak-Durchsatzes:

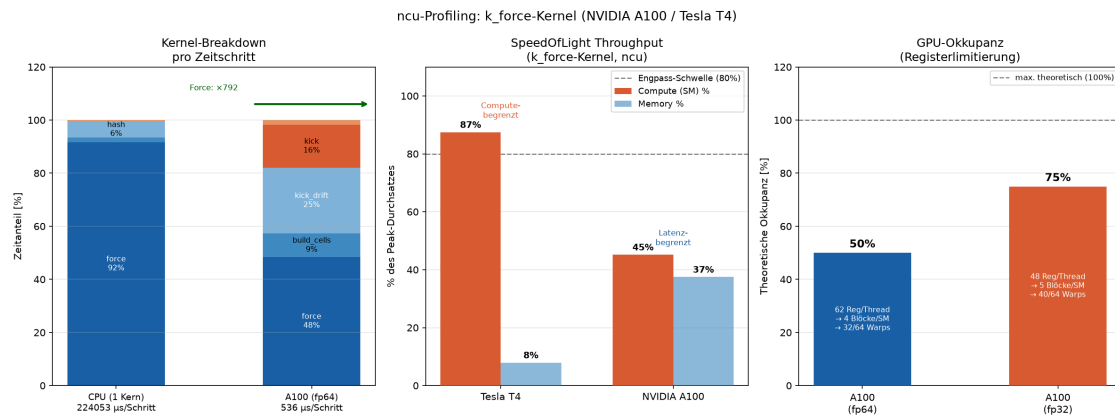


Abbildung 17: ncu-Profilng des `k_force`-Kernels. Links: Kernel-Zeitanteile pro Schritt (CPU vs. A100 fp64). Mitte: SpeedOfLight-Throughput – T4 ist compute-bound (87 %), A100 ist latenz-limitiert (45 % Compute, 37 % Memory). Rechts: Theoretische Okkupanz durch Registerlimitierung. Skript: `scripts/python/plot_ncu.py`.

Befund A100.

- Compute-Throughput: **45 %** (nicht compute-bound).
- Memory-Throughput: **37 %** (nicht vollständig memory-bound).
- Schlussfolgerung: Der Kernel ist *latenz-limitiert* – Threads warten auf Daten aus dem DRAM, können nicht rechnen, und der Scheduler hat nicht genug alternative Warps um die Latenz zu verstecken.

Befund Tesla T4.

- Compute-Throughput: **87 %** – klar compute-bound.
- Memory-Throughput: **8 %** – sehr gering.

Der Unterschied: Die T4 hat viel weniger fp64-Rechenleistung ($\approx 0,25$ TFLOPS vs. 9,7 TFLOPS beim A100), aber dieselbe Kernel-Struktur. Die begrenzte Rechenkapazität der T4 wird voll ausgelastet; beim A100 ist die Bandbreite der Engpass – aber nur zu 37 % ausgenutzt.

11.2 Okkupanz: Registerlimitierung

Okkupanz-Rechnung (fp64, A100)

Register pro Thread: 62 (gemessen mit `nvcc -ptxas-info`). Register-Budget pro SM: 65 536. Maximale Blöcke/SM: $\lfloor 65536 / (62 \times 256) \rfloor = 4$. Warps/SM bei 4 Blöcken: $4 \times 8 = 32$. Max Warps/SM (A100): 64.
⇒ **Okkupanz** = $32/64 = 50\%$.

Mit nur 50 % Okkupanz hat der Scheduler nur halb so viele Warps verfügbar, um Speicherlatenz zu verstecken. Bei fp32 sinkt der Registerbedarf auf ≈ 48 Register/Thread, was 5 Blöcke/SM ermöglicht und die Okkupanz auf 62 % erhöht.

Naheliegender Hebel. Um die Okkupanz zu verbessern, müsste die Register-Allokation reduziert werden, damit mehr Warps gleichzeitig je SM laufen und die Speicherlatenz besser versteckt wird. Der direkteste Weg dazu ist das Pragma `__launch_bounds__(maxThreads, minBlocksPerSM)`, das `ptxas` zwingt, die Register je Thread so weit zu begrenzen, dass `minBlocksPerSM` Blöcke gleichzeitig Platz finden. Ob dieser Hebel *tatsächlich* Leistung bringt, lässt sich direkt messen – der nächste Abschnitt tut das.

11.3 Experiment: Occupancy-Hebel via `__launch_bounds__`

Vier Builds wurden bei sonst identischem Code verglichen ($1414 \times 1414 \approx 2$ Mio Atome, fp64, Blockgröße 256, A100): **base** ohne Begrenzung sowie **lb4/lb6/lb8** mit `minBlocksPerSM = 4/6/8`. Die Absolutwerte dieser Messreihe (base: 3435 Matom/s) stammen aus einer separaten Kampagne und liegen $\approx 10\%$ unter dem Headline-Benchmark aus Abschnitt 9.2; für die Aussage zählt der Vergleich der Varianten untereinander.

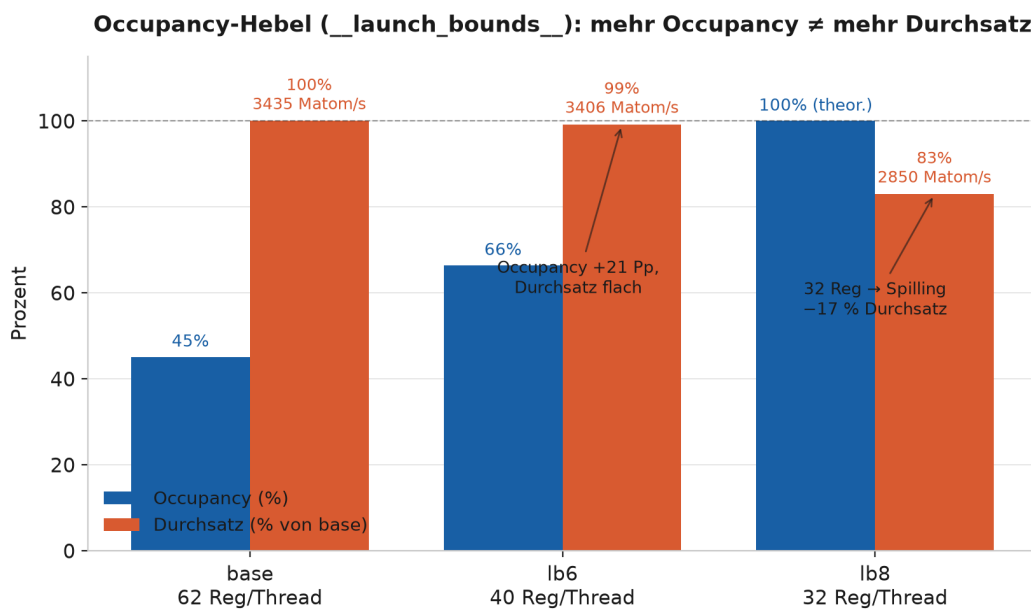


Abbildung 18: Occupancy-Hebel via `__launch_bounds__`. Blau: per ncu gemessene Okkupanz; orange: Durchsatz relativ zur **base**-Variante. **lb4** ist nicht separat dargestellt, da es (siehe Tabelle unten) mit 62 Reg/Thread identisch zu **base** ist und keine eigene Wirkung zeigt. Die Begrenzung hebt die Okkupanz von 45 % (base, 62 Reg) auf 66 % (lb6, 40 Reg) – der **Durchsatz bleibt jedoch flach** (3435 $\rightarrow$ 3406 Matom/s). Bei lb8 (32 Reg, theoretisch volle Okkupanz) kippt es: **Register-Spilling** kostet 17 % Durchsatz. Skript: `scripts/python/plot_launch_bounds.py`.

Variante	Reg/Thread	Okkupanz	force-Kernel	Durchsatz
base	62	45,1 % (ncu)	276 μ s	3435 Matom/s
lb4	62	= base	= base	3435 Matom/s
lb6	40	66,4 % (ncu)	262 μ s	3406 Matom/s
lb8	32	$\sim 100\%$ (theor.)	387 μ s	2850 Matom/s

Interpretation.

- **lb4 ist wirkungslos:** Der Kernel braucht ohnehin nur 62 Register, was bereits 4 Blöcke/SM erlaubt – die Begrenzung ändert nichts.
- **lb6 ist der Kernbefund:** Die Register sinken von 62 auf 40, die *gemessene* Okkupanz steigt von 45 % auf 66 % – der Hebel greift also nachweislich. Trotzdem bleibt der Durchsatz im Messrauschen unverändert. *Mehr Okkupanz übersetzt sich nicht in mehr Leistung.*

- **lb8 zeigt die Kehrseite:** Bei 32 Registern tritt Register-Spilling auf, der Kraft-Kernel wird $\approx 40\%$ langsamer (Gesamtdurchsatz -17%) – der klassische Trade-off zu aggressiver Registerbegrenzung.

Diagnose experimentell bestätigt

Die Okkupanz ließ sich gezielt von 45 % auf 66 % anheben, ohne dass der Durchsatz folgte. Das **bestätigt die ncu-Diagnose**: Der Kraft-Kernel ist latenz- bzw. bandbreitengebunden, *nicht* occupancy-limitiert. Zusätzliche Okkupanz ist erzeugbar, aber wirkungslos – der Engpass liegt am Datenlayout (SoA-Bandbreite, Zugriffskoaleszenz), nicht an der Zahl aktiver Warps. Ein vollwertiges Negativergebnis: der laut Diagnose richtige Hebel wurde umgesetzt, vermessen und als wirkungslos nachgewiesen.

11.4 Weitere Optimierungsversuche: Launch-Overhead und Präzision

Die ncu-Diagnose (Abschnitt 11) nennt zwei naheliegende Hebel neben der Okkupanz: den Host-seitigen Kernel-Start-Overhead und die Rechenpräzision. Beide wurden zusätzlich zum Okkupanz-Experiment aus Abschnitt 11.3 gezielt getestet und gemeinsam mit diesem in einer konsolidierten Messreihe verglichen (vier Varianten, sonst identischer Code, ≈ 2 Mio Atome, fp64).

GPU-Optimierung (fp64, 2 Mio Atome, A100): Occupancy hebbar — Leistung nicht

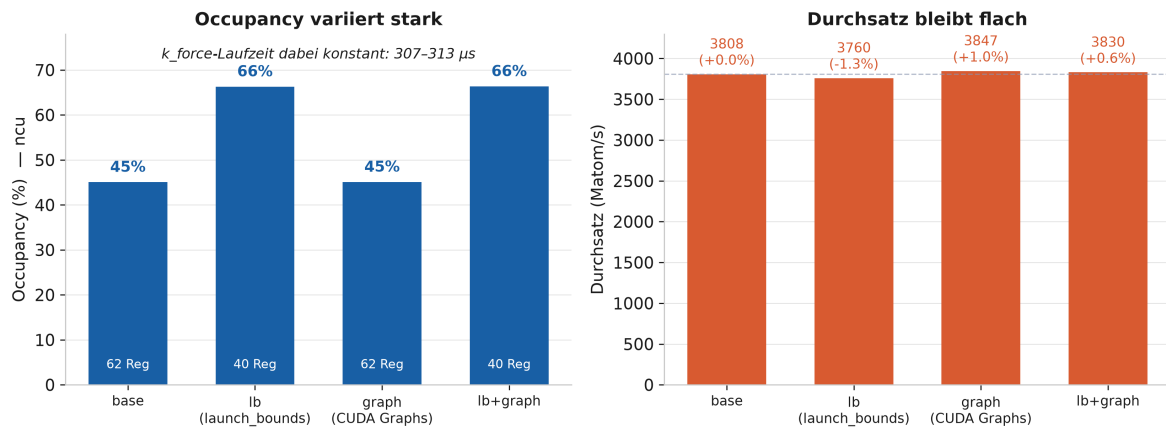


Abbildung 19: Vier GPU-Varianten im Vergleich: base, lb (`__launch_bounds__`, 40 statt 62 Register), graph (CUDA Graphs) und lb+graph. Links: die Okkupanz lässt sich durch lb von 45 % auf 66 % anheben. Rechts: der Gesamtdurchsatz bleibt über alle Varianten hinweg flach bei ≈ 3800 Matom/s ($\pm 1\%$, im Bereich der Messstreuung). Skript: `scripts/python/plot_cuda_opt.py`, Daten: `results/cuda_opt_results.csv`.

Launch-Overhead (CUDA Graphs). Pro Zeitschritt werden fünf Kernel gestartet; über tausende Schritte summiert sich der Host-seitige Start-Overhead. Mit CUDA Graphs wird die feste Kernel-Sequenz einmalig aufgezeichnet (`cudaStreamBeginCapture`) und anschließend pro Schritt mit einem einzigen `cudaGraphLaunch` abgespielt – die Physik bleibt dabei unverändert. Der resultierende Gewinn liegt bei $\approx 1\%$ und ist damit vernachlässigbar. Der Energie-Drift ist in allen vier Varianten identisch.

Präzision (fp32 statt fp64). Der Wechsel auf einfache Genauigkeit (Abschnitt 9.2) bringt Faktor $1,58\times$ statt der theoretisch möglichen $2\times$. Der Gewinn stammt im Wesentlichen aus der halbierten Datenmenge, nicht aus mehr Rechenleistung – ein weiterer Beleg für die Bandbreiten- bzw. Latenzbindung. Für die Kaskaden-Ensembles (Teil II) bleibt es dennoch bei fp64: Bindungsbrüche hängen an feinen Abstandsdifferenzen (Bruchschwelle $1,15 L_0$), und der Energie-Drift verschlechtert sich unter fp32 deutlich ($0,94\%$ statt $0,22\%$).

Fazit: drei Hebel, ein Engpass

Höhere Okkupanz, reduzierter Launch-Overhead und reduzierte Präzision verbessern die Leistung jeweils nicht wesentlich und weisen konsistent auf denselben Engpass hin: die Speicher-Latenz und -Bandbreite des `k_force`-Kernels. Eine weiterführende Optimierung müsste am Datenlayout und an der Zugriffskoaleszenz ansetzen statt an der Auslastung. Damit bestätigt die GPU-Analyse denselben limitierenden Faktor, der bereits die CPU- und Skalierungsergebnisse dieser Arbeit bestimmt.

12 Fazit Teil I

Der durchgehende rote Faden: Speicherbandbreite.

- **Einzelkern (Compiler):** `-O3` = $2\times$. `-Ofast` hilft nicht (und verletzt Korrektheit).
- **Strong Scaling:** U-förmig. Superlinear bei 128 Kernen durch Cache-Effekt (Datensatz pro Rank unterschreitet die 4-MB-L3-Schwelle).
- **Weak Scaling:** Bricht ab 32 Kernen ein – gesättigte Knotenbandbreite.
- **Optimierungen:** Non-blocking Halo und Re-Sorting bringen $+0,8\%$ – die Kommunikation ist nicht der Engpass.
- **GPU:** A100 ist $731\times$ schneller als 1 Kern, läuft aber nur mit 45% Compute- und 37% Memory-Auslastung – latenz-limitiert.
- **Occupancy-Hebel (`__launch_bounds__`):** Okkupanz gezielt auf 66% angehoben – Durchsatz bleibt flach. Bestätigt die Bandbreiten-/Latenz-Diagnose experimentell statt nur per Profiler.
- **CUDA Graphs & Präzision:** Reduzierter Launch-Overhead ($+1\%$) und fp32 statt fp64 ($1,58\times$ statt theor. $2\times$) bringen beide keinen wesentlichen Zusatzgewinn – drei unabhängige Hebel, ein Engpass.

Teil II

Power Law – Skalenfreie Defektverteilung

13 Theoretischer Hintergrund

13.1 Was ist ein Potenzgesetz?

Eine Wahrscheinlichkeitsverteilung $p(n)$ folgt einem Potenzgesetz, wenn

$$p(n) \propto n^{-(\alpha+1)}, \quad n \geq n_{\min} > 0, \quad (4)$$

mit einem Exponenten $\alpha > 0$. In der Literatur zur Strahlenschaden-Physik wird die Konvention $F(n) = A n^{-S}$ verwendet (Häufigkeit statt Wahrscheinlichkeitsdichte), mit $S = \alpha + 1$. Charakteristisch für Potenzgesetze ist das Fehlen einer ausgezeichneten Skala: es gibt keine “typische” Clustergröße. Das ist das Kennzeichen *selbstorganisierter Kritikalität* – verwandte Phänomene sind Erdbebengrößen, Waldbrände, Netzwerk-Knotengrade.

CCDF – Complementary Cumulative Distribution Function. Direkt aus den Daten schätzbar ist die *komplementäre kumulative Verteilungsfunktion*

$$P(N \geq n) = \int_n^\infty p(n') dn' \propto n^{-S+1}, \quad (5)$$

die im log-log-Plot als Gerade mit Steigung $-(S - 1)$ erscheint. Die CCDF hat gegenüber einem Histogramm den Vorteil, keine Binning-Entscheidungen zu benötigen und den Schwanz der Verteilung weniger zu glätten.

13.2 Sand et al. (2013): Potenzgesetz in 3D-Wolfram

Sand et al. führten vollständige MD-Simulationen von 150-keV-Kollisionskaskaden in Wolfram durch (PARCAS-Code, ≈ 100 Kaskaden, drei verschiedene interatomare Potenziale). Wolfram (W) ist das Kandidatmaterial für die erste Wand in Fusionsreaktoren wegen hoher Schmelztemperatur, guter Wärmeleitfähigkeit und geringer Neutronenaktivierung.

Schlüsselbefunde.

- Die Größenverteilung der Zwischengitteratom-Cluster (SIA-Cluster) folgt über fast drei Größenordnungen dem Potenzgesetz:

$$F(n) = A n^{-S}, \quad A = 7,45 \pm 1,52, \quad S = 1,63 \pm 0,07 \quad (6)$$

- In ≈ 100 Kaskaden wurde *keine einzige* Subkaskaden-Fragmentierung beobachtet. Das Schmelzvolumen bleibt kompakt (zusammenhängend, $\sim 30\,000$ Atome, mittlere Energiedichte ≈ 4 eV/Atom).
- **Kontrast zu Eisen:** In Fe fragmentieren Kaskaden bereits ab ≈ 50 keV. W-Atome sind schwerer, Kollisionsquerschnitte größer, Rückstoßatome langsamer – deshalb kompaktere Schmelzbereiche.
- Die “fraktale Natur” der Kaskade (selbstähnliche Verzweigungsstruktur) wurde direkt visualisiert. Sie ist die geometrische Ursache des Potenzgesetzes.

Versetzungsschleifen-Charaktere. Große Schleifen (> 80 SIAs) haben durchgehend $\mathbf{b} = \frac{1}{2}\langle 111 \rangle$; kleinere zeigen zu etwa gleichen Teilen beide Burgers-Vektoren ($\frac{1}{2}\langle 111 \rangle$ und $\langle 100 \rangle$). Die Wahrscheinlichkeit einer Schleife mit > 150 Defekten beträgt ca. 3 % – in guter Übereinstimmung mit Experimenten.

13.3 De Backer et al. (2016): Subkaskaden und das fragmentierte Regime

De Backer et al. erweitern die Analyse auf hohe PKA-Energien, bei denen die Kaskade in räumlich getrennte *Subkaskaden* zerfällt. Sie verwenden eine Subdomain-Dekompositionsmethode kombiniert mit der Binärkollisions-Näherung.

Fragmentierungsenergie E_{fr} . In allen 20 untersuchten Metallen (u.a. W, Fe, Be, Zr) existiert eine charakteristische Energie E_{fr} , oberhalb derer Subkaskaden entstehen. Die mittlere Anzahl der Sub-Domänen wächst als:

$$\nu(E) = \begin{cases} 1 & E \leq 2^{-1/P} E_{\text{fr}} \\ 2 (E/E_{\text{fr}})^P & E \geq 2^{-1/P} E_{\text{fr}} \end{cases} \quad (7)$$

mit $P \in [0,8; 1,2]$. Für Wolfram liegt $E_{\text{fr}} \approx 160$ keV; über alle untersuchten Metalle reicht die Schwelle von 2 bis ≈ 500 keV. Die 2D-Kaskaden dieser Arbeit zeigen die Morphologie des nicht-fragmentierten Regimes – ein zusammenhängendes Schadensgebiet statt räumlich getrennter Subkaskaden (Abb. 20) – und liegen damit unterhalb der (modellabhängigen) Fragmentierungsschwelle, konsistent mit dem sauberen Potenzgesetz, das hier beobachtet wird.

Generalisiertes Skalierungsgesetz. Oberhalb von E_{fr} ergibt sich ein modifiziertes Skalierungsgesetz:

$$G(n, E) \sim n^{-S} \left[1 - \left(\frac{n}{n_{\text{fr}}} \right)^{1+S-\frac{3}{2}Q(E)} \right], \quad E > E_{\text{fr}} \quad (8)$$

Der Korrekturterm in der eckigen Klammer dämpft große Cluster: oberhalb von E_{fr} ist die maximale Subkaskaden-Größe durch E_{fr} begrenzt, unabhängig von der Gesamtenergie. Dies erklärt die kontraintuitive Beobachtung, dass *hochenergetische Kaskaden weniger große Defektcluster erzeugen als niederenergetische*.

Verbindung zur eigenen Simulation. Die 2D-Ergebnisse (Abschnitt 16) zeigen dasselbe skalenfreie Verhalten wie Sand et al. – allerdings im nicht-fragmentierten Regime, mit niedrigerem Exponenten (2D vs. 3D erwartet eine Verschiebung). Für höhere E_{PKA} -Werte wäre analog zu De Backer ein Finite-Size-Cutoff durch Gittergrenzen zu erwarten.

14 Ensemble-Setup

Parameter.

- 1000 unabhängige Kaskaden: serieller Code, 128 Instanzen gleichzeitig (`xargs -P 128`).
- Variiert: Einschlagort (zufällig), Einschlagwinkel (uniform in $[0, 2\pi)$), PKA-Energie (log-uniform in $[200; 2500]$).
- Gitter: 250×250 Atome, $dt = 3 \times 10^{-4}$, 12 000 Schritte.
- Cluster-Erkennung: Union-Find mit Schwellwert-Verschiebung $> 0,5 L_0$ und Verbindungsradius $< 1,5 L_0$.

Zwei Varianten: Heal vs. No Heal.

- **Heal:** Bindungen können sich nach dem Bruch wieder schließen, falls die Atome sich re-annähern.
- **No Heal:** Einmal gebrochene Bindungen bleiben permanent offen.

Ergebnis: 1000 Läufe mit Healing ergeben **2391 gepoolte Cluster**; 1000 Läufe ohne Healing ergeben **2335 gepoolte Cluster**. Die Zahl der Cluster ist fast identisch – das Healing beeinflusst die Statistik kaum, weil die Fragmentierung in der schnellen ballistischen Phase festgelegt wird, bevor das Healing-Fenster greift. Abbildung 20 zeigt die Schadensmorphologie beider Varianten im direkten Vergleich.

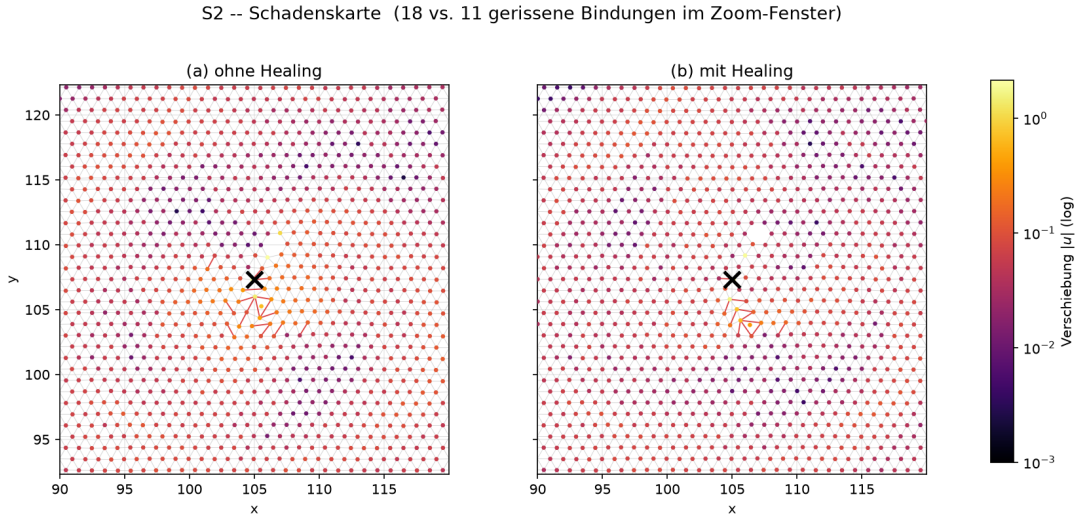


Abbildung 20: Schadenskarte einer Kaskade (Zoom $\pm 15 L_0$ um den Einschlag, Lauf 999 aus dem Ensemble). Farbe = Verschiebung $\log|u|$; rote Segmente = Bindungen, deren aktueller Abstand die Bruchschwelle $1,15 L_0$ überschreitet (aus den Endzustands-CSVs geometrisch rekonstruiert, da keine separate Bindungsliste vorliegt). Links: ohne Healing (18 gerissene Bindungen im Zoomfenster). Rechts: mit Healing (11). Beide zeigen die typische Morphologie: dichter Kern plus verzweigte Ausläufer. Healing dünnt den Schaden etwas aus, ändert aber die Skalierungsstruktur nicht. Skript: `scripts/python/plot_damage_map.py`.

15 Fit-Methodik: Maximum-Likelihood-Estimation

15.1 Clauset et al. – MLE statt Histogramm-Fit

Ein linearer Fit im log-log-Histogramm ist *kein* statistisch valider Schätzer für den Potenzgesetz-Exponenten (Bias durch Binning, falsche Fehlerstruktur). Stattdessen kommt Maximum-Likelihood-Estimation (MLE) nach Clauset et al. (2009) zum Einsatz. Für ganzzahlige Daten mit unterem

Cutoff $n_{\min}$ ist der MLE-Schätzer:

$$\hat{S} = 1 + n_{\text{Stichprobe}} \left[\sum_{i=1}^{n_{\text{Stichprobe}}} \ln \frac{x_i}{x_{\min} - 1/2} \right]^{-1}, \quad (9)$$

kombiniert mit dem Kolmogorov-Smirnov-Test zur Wahl des optimalen $x_{\min}$.

15.2 Die $x_{\min}$ -Falle

Naive Minimierung des KS-Abstands kann in einem ersten Auswertungsversuch in den Verteilungsschwanz “fliehen” und einen Artefakt-Exponenten $S \approx 29,6$ bei $x_{\min} \approx 2571$ liefern (weit außerhalb des Datenbereichs). Die KS-Minimierung findet dann den Bereich hinter der “Klippe” (Finite-Size-Cutoff) als besten Fit – aber nur für die drei verbleibenden Punkte dort.

Lösung. $x_{\min}$ -Scan nur im datenreichen Rumpf:

- Minimum: $\geq 5\%$ der Datenpunkte über $x_{\min}$ (mind. 50 Punkte),
- Maximum: $x_{\min} < 90$ -Perzentil des Datensatzes.

Damit wird der Artefakt ausgeschlossen und der eigentliche Potenzgesetz-Rumpf robust gefittet.

16 Ergebnisse: Power Law in 2D

Abbildung 21 zeigt zunächst die statistische Konvergenz: erst ab $N \sim 50$ gepoolten Läufen wird der Potenzgesetz-Rumpf der CCDF sichtbar – die 1000 Läufe des Ensembles sind also nicht Luxus, sondern Voraussetzung.

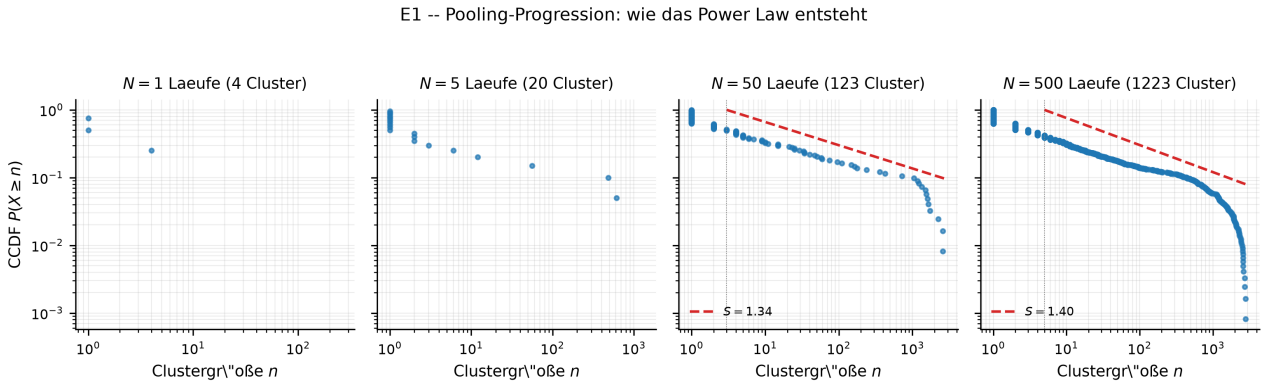


Abbildung 21: Pooling-Progression. Von $N = 1$ bis $N = 500$ werden zufällig gezogene Läufe gepoolt und die CCDF der Clustergrößen geplottet. Bei $N = 1$ liegt genau ein Punkt. Mit wachsendem N füllt sich die Verteilung; ab $N \sim 50$ ist die Potenzgesetzkurve sichtbar. *Statistisch aussagekräftige Aussagen erfordern ausreichend viele Kaskaden.*

Schlüsselergebnis.

Variante	Läufe	Cluster	$\hat{S}$	$x_{\min}$
Mit Healing	1000	2391	$1,41 \pm 0,03$	4
Ohne Healing	1000	2335	$1,38 \pm 0,03$	4
Sand et al. 3D	–	–	$1,63 \pm 0,07$	–

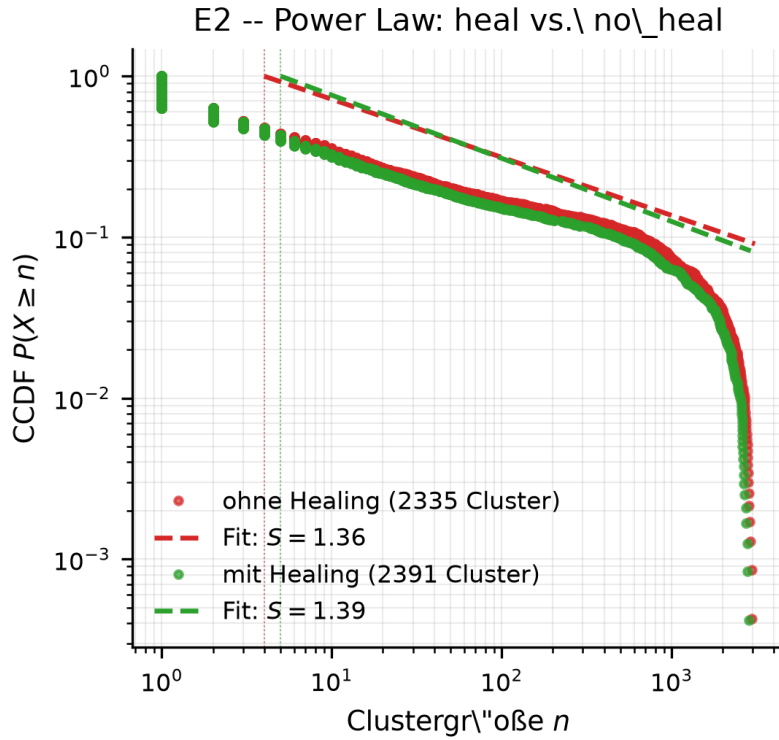


Abbildung 22: Power Law – Overlay Heal vs. No Heal (je 1000 Läufe, 2391 bzw. 2335 Cluster). CCDF im log-log-Plot. Beide Kurven fallen über fast drei Größenordnungen als Gerade: skalenfreies Potenzgesetz. Der MLE-Fit liefert $S = 1,41$ (Heal) und $S = 1,38$ (No Heal) bei $x_{\min} = 4$. Referenzlinie: Sand et al. (2013) 3D-Wolfram mit $S = 1,63$.

E3 -- Clustergrößen-Verteilung (PDF + CCDF)

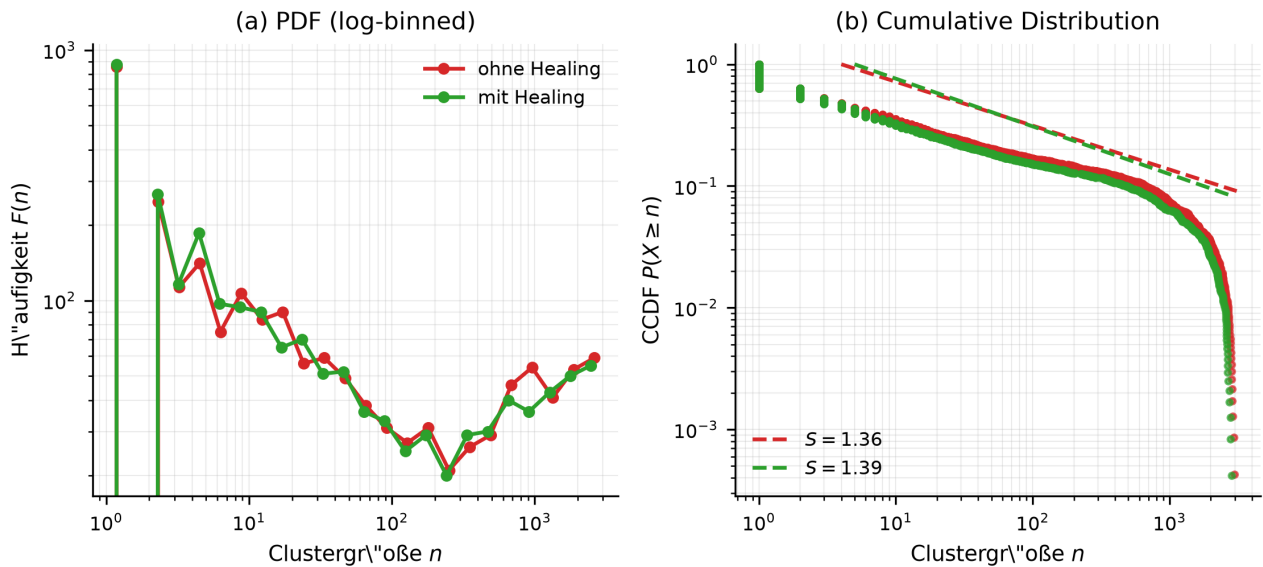


Abbildung 23: Duale Darstellung: PDF (log-binned Histogramm, links) und CCDF (rechts), jeweils im Vergleich mit und ohne Healing. Der Finite-Size-Cutoff am oberen Ende (Klippe) ist in beiden Darstellungen sichtbar: das endliche Gitter setzt eine obere Grenze für die Clustergröße.

17 Diskussion

17.1 Warum $S < 1,63$?

Der gemessene Exponent $S \approx 1,4$ liegt systematisch unter dem 3D-Wert $S = 1,63$. Drei Effekte tragen dazu bei:

1. **Dimension (2D vs. 3D):** In 2D hat die Kaskade einen kleineren Phasenraum für Fragmentierung und weniger Wege zur Energieverteilung. Das führt tendenziell zu mehr großen Clustern (flacherer CCDF, kleineres S).
2. **Modellvereinfachung:** Das Harmonic-Feder-Modell ohne reale Potenziale (ZBL, EAM) erfasst nicht die korrekte energieabhängige Streuquerschnittsstruktur. In realen Systemen hängt die Kaskadenmorphologie stark vom interatomaren Potenzial ab.
3. **Endliche Systemgröße:** Das 250×250 -Gitter setzt einen Finite-Size-Cutoff bei großen Clustern (sichtbar als Klippe in Abb. 23). MLE mit $x_{\min} = 4$ fittet nur den Rumpf; der Schwanz ist nicht für die Schätzung verantwortlich.

17.2 Robustheit gegen Healing

Der Exponent S ändert sich durch Healing von 1,38 auf 1,41 – eine Verschiebung von nur $\approx 2\%$. Die Erklärung ist konsistent mit dem Mechanismus: die Kaskadengeometrie (und damit die Clustergrößen) wird in der schnellen *ballistischen Phase* festgelegt. Healing (langsame Annäherung von Atompaaren) greift erst danach und betrifft hauptsächlich kleine Cluster an der Peripherie. Die Skalierungsstruktur ist robust.

17.3 Verbindung zur De Backer-Theorie

De Backer et al. sagen voraus, dass große Cluster oberhalb von E_{fr} durch das modifizierte Skalierungsgesetz (8) unterdrückt werden. Im hier verwendeten Modell entspricht der Finite-Size-Cutoff einem analogen Effekt: das Gitter setzt eine maximale Subkaskaden-Größe. Für höhere PKA-Energien würde das modifizierte Skalierungsgesetz relevant – das ist die natürliche Erweiterungsfrage.

17.4 Ausblick und offene Fragen

- **Bootstrap-Konfidenzintervalle:** Per `powerlaw`-Paket (Alstott et al., 2014) wäre ein rigoroseres Konfidenzintervall zu bestimmen.
- **Subkaskaden-Schwellwert:** Bei höherer E_{PKA} sollte gemäß De Backer ein Übergang zu einer gedämpften Verteilung sichtbar werden.
- **3D-Erweiterung:** Das Potenzgesetz in 3D mit realen Potenzialen (ZBL + Einbettungsenergie-Modell) zu reproduzieren, wäre der nächste Schritt zum direkten Vergleich mit Sand et al.
- **GPU-Ensemble:** Mit dem A100 (731-fache Beschleunigung) könnten 10 000 statt 1 000 Kaskaden in derselben Rechenzeit simuliert werden – der statistische Fehler des Exponenten würde von $\pm 0,03$ auf $\pm 0,01$ sinken.

18 Machbarkeitsstudie: Skalierung des Ensembles

Könnte man den Exponenten S durch einen *deutlich größeren* Lauf schärfer bestimmen – mehr Kaskaden (engeres Konfidenzintervall) und größere Gitter (mehr Dekaden im Potenzbereich)? Diese Frage wurde unter einer harten Randbedingung untersucht: ≈ 2000 verbleibende CPU-Stunden auf Noctua 2, der Lauf musste also im ersten Versuch gelingen. Daraus folgte das Prinzip „**erst messen, dann rechnen**“ – jeder Budgetkennwert wurde an einer kleinen Probe bestimmt, statt ihn zu schätzen. Abbildung 24 fasst die beiden entscheidenden Messungen zusammen.

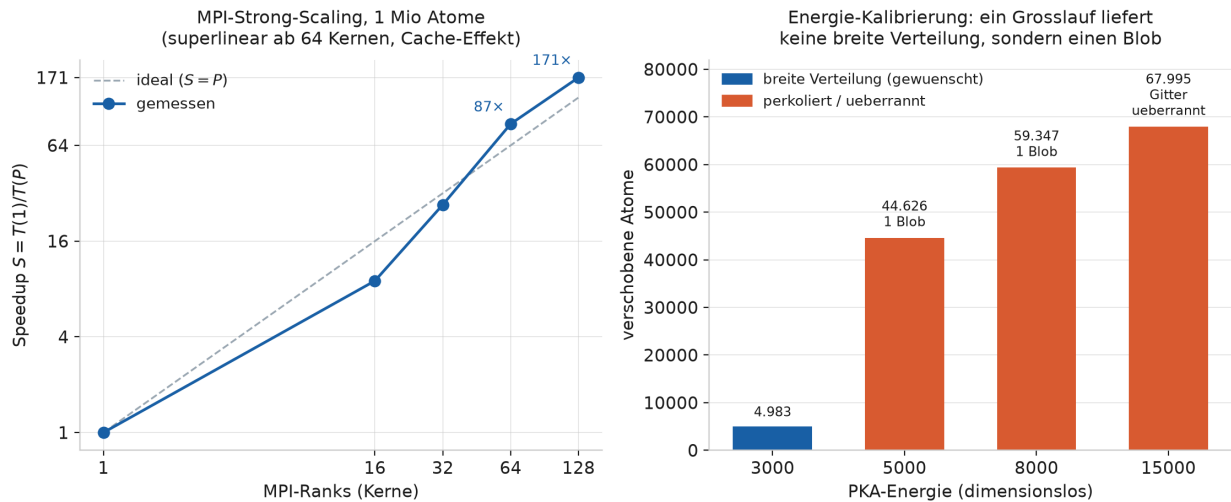


Abbildung 24: Voruntersuchung zum skalierten Ensemble-Lauf. **Links:** MPI-Strong-Scaling eines einzelnen 1000×1000 -Laufs (1 Mio Atome). Die Skalierung ist bei 64 und 128 Kernen *superlinear* ($87\times$ bzw. $171\times$) – derselbe Cache-Effekt wie im Hauptteil (Abschnitt 5). Daraus folgt die Produktionswahl 128 Kerne/Lauf: ein Lauf belegt einen Knoten allein und behält die superlineare Effizienz. **Rechts:** Energie-Kalibrierung. Mit steigender PKA-Energie verschmilzt der Schaden zu *einem* perkolierten Cluster (größter Cluster $\approx$ alle verschobenen Atome); ab $\approx 15\,000$ überrennt er das Gitter. Ein einzelner Großlauf liefert also keine breite Größenverteilung. Skript: `scripts/python/plot_ensemble_scaling.py`.

Budget-Probe (seriell). Eine voll gepackte Welle von 128 gleichzeitigen Läufen (realistisch, da sie sich die Knotenbandbreite teilen) zeigt *überlineare* Kosten mit der Gittergröße ($\approx N_x^{3,5}$ statt N_x^2) – wieder die geteilte Speicherbandbreite als Engpass:

Gitter	Atome	Wandzeit 128er-Welle	Bytes/Lauf
250×250	62 500	39,97 s	1,63 MB
400×400	160 000	209,07 s	4,23 MB

MPI-Skalierung bei 1 Mio Atomen. Der linke Plot in Abb. 24 basiert auf folgenden Wandzeiten; die Effizienz übersteigt bei 64/128 Kernen 100 % (cache-resident):

Kerne	Wandzeit	Speedup	Effizienz
1	533,56 s	1,0	100 %
16	59,43 s	9,0	56 %
32	19,70 s	27,1	85 %
64	6,11 s	87,3	136 %
128	3,12 s	171,0	134 %

Für den Großlauf wurde die MPI-Version um eine `MPI_Gatherv`-Ausgabe des per-Atom-Endzustands erweitert (bislang gab sie nur aggregierte Größen aus). Die Validierung bestätigte die Korrektheit: MPI mit 1 vs. 4 Rängen ist **bit-identisch**; seriell vs. MPI weicht erwartungsgemäß pointweise ab (chaotische Trajektorien, unterschiedliche Gleitkomma-Summationsreihenfolge), aber die *Verteilung* – das einzig Relevante für das Power Law – ist identisch.

Entscheidung: Lauf nicht durchgeführt. Die Infrastruktur war lauffähig und validiert, und das Budget hätte rund 1500 Läufe getragen. Entscheidend dagegen sprach der *wissenschaftliche* Befund der rechten Bildhälfte: Ein größeres Gitter mit höherer Energie erzeugt nicht mehr Dekaden, sondern bei hoher Energie einen einzelnen perkolierten Blob. Der erwartete Erkenntnisgewinn gegenüber dem bestehenden Ergebnis ($S = 1,38/1,41$) rechtfertigte Aufwand und Risiko im gegebenen Rahmen nicht. Das ist dieselbe Methodenlehre wie bei der GPU-Analyse: **erst die richtige Messgröße bestimmen, dann Rechenzeit investieren.**

19 Fazit Teil II

Skalenfreies Verhalten emergiert aus dem Mechanismus. Das 2D-Dreiecksgitter-Modell – ohne Quantenmechanik, ohne reale Wolfram-Potenziale – reproduziert das skalenfreie Potenzgesetz der Defektclustergrößen mit einem Exponenten $S \approx 1,4$. Das Ergebnis ist:

- **Robust gegen Healing:** Nur 2 % Verschiebung.
- **Konsistent mit Sand et al.:** Gleiche qualitative Struktur (skalenfreies Potenzgesetz), niedrigerer Exponent aus 2D/Modell-Gründen.
- **Jenseits des De Backer-Regimes:** Im nicht-fragmentierten Bereich, d.h. das generalisierte Skalierungsgesetz (8) wird noch nicht aktiv.

Methodische Lehre. Die $x_{\min}$ -Falle beim KS-basierten Fit – Artefakt $S \approx 29,6$ – zeigt, dass Potenzgesetz-Fits ohne Daten-Rumpf-Einschränkung täuschen. MLE nach Clauset et al. mit expliziter $x_{\min}$ -Validierung ist die einzig zuverlässige Methode.

A Anhang: Skripte und Reproduzierbarkeit

Python-Plot-Skripte (scripts/python/).

Skript	Erzeugt	Abbildung
analyze_scaling.py	strong_scaling.png ¹	8, 10, 11
analyze_weak.py	weak_scaling.png ¹	13
analyze_compiler_opt.py	compiler_opt.png ¹	7
analyze_opt_compare.py	opt_compare.png ¹	14
analyze_ensemble.py	CCDF-Plots	21, 23
plot_damage_map.py	vid_S2_damage.png	20
overlay_ccdf.py	Overlay-Plot	22
plot_tts.py	tts.png	15
plot_ncu.py	ncu_profiling.png	17
plot_launch_bounds.py	launch_bounds_plot.png	18
plot_gpu_saturation.py	gpu_saturation.png	16
plot_ensemble_scaling.py	ensemble_scaling.png	24
plot_cache_argument.py	cache_argument.png	9

```
# Strong Scaling (1+2 Knoten):
python3 analyze_scaling.py results/mpi_strong_scaling_1node.csv \
    results/mpi_strong_scaling_2node.csv

# Cache-Argument:
python3 plot_cache_argument.py results/mpi_strong_scaling_1node.csv \
    results/mpi_strong_scaling_2node.csv

# TTS-Plot (Daten sind hardcoded aus Benchmark-Outputs):
python3 plot_tts.py

# ncu-Profilng (Daten hardcoded aus ncu_report.csv + Benchmark-Outputs):
python3 plot_ncu.py

# Schadenskarte (rekonstruiert Bindungen geometrisch aus den Endzuständen):
python3 plot_damage_map.py results/ensemble_no_heal_run_0999_state_final.csv \
    results/ensemble_run_0999_state_final.csv
```

¹Im Bericht unter einem umbenannten Dateinamen mit H/E/M-Präfix eingebunden (z. B. H1_strong_scaling.png statt strong_scaling.png). Diese Version wurde für die Abbildungen im Bericht separat erzeugt und entspricht nicht notwendig 1:1 dem aktuellen Stand des hier gelisteten Skripts.

Hardware. Alle CPU-Messungen auf *Noctua 2* (PC2 Paderborn): EPYC 7763, 128 Kerne/Knoten, 256 GB RAM, HDR InfiniBand. GPU-Messungen auf demselben Cluster: NVIDIA A100-SXM4-40GB, CUDA 13.2. T4-Messungen auf einem separaten System (sm_75, CUDA 12.x).

Literatur

- [1] A. E. Sand, S. L. Dudarev, K. Nordlund: *High-energy collision cascades in tungsten: Dislocation loops structure and clustering scaling laws*. EPL (Europhysics Letters) 103 (2013) 46003.
- [2] A. De Backer, A. Sand, K. Nordlund, L. Luneville, D. Simeone, S. Dudarev: *Subcascade formation and defect cluster size scaling in high-energy collision events in metals*. EPL 115 (2016) 26001.
- [3] A. Clauset, C. R. Shalizi, M. E. J. Newman: *Power-law distributions in empirical data*. SIAM Review 51(4) (2009), 661–703.